

Ruben Apablaza Muñoz



PROYECTO  
LABS  
CIBERSEGURIDAD

LAB9  
Cracking de Hashes

— 0Zone —

## ⚠ **AVISO LEGAL Y DESCARGO DE RESPONSABILIDAD** ⚠

Este documento tiene fines **estrictamente educativos y de formación profesional** en el área de la ciberseguridad. El autor no se hace responsable por el mal uso de la información, scripts o comandos aquí descritos.

- **Entornos de prueba**

Se recomienda ejecutar este laboratorio exclusivamente en entornos controlados (Máquinas Virtuales o Contenedores). Nunca ejecutes scripts o comandos desconocidos en servidores de producción sin antes probarlos y entender su funcionamiento.

- **Responsabilidad del usuario**

El uso de herramientas de nivel administrativo (`sudo`) conlleva riesgos. El lector es el único responsable de cualquier pérdida de datos o interrupción del servicio que pudiera derivarse de la aplicación de este manual.

- **Ética profesional**

Las técnicas de análisis aquí descritas deben ser utilizadas siempre bajo un marco legal y con la autorización debida.



## Contenido

|                                                                   |    |
|-------------------------------------------------------------------|----|
| Introducción .....                                                | 4  |
| Requisito de operación.....                                       | 5  |
| 1. Obtener hash real desde Ubuntu Server .....                    | 8  |
| Paso 1: Extraer el hash de un usuario de prueba .....             | 8  |
| 2. Compartir archivos por SCP .....                               | 10 |
| METODO SCP (SSH) .....                                            | 10 |
| 3. Romper el hash (Kali) .....                                    | 16 |
| 3.1 Confirmar el tipo de hash .....                               | 17 |
| 3.2. Intentar romper el hash con formato yescrypt .....           | 22 |
| 3.2.1 Opción John the Ripper .....                                | 22 |
| 3.2.2 Opción simple: Hashcat.....                                 | 23 |
| 3.2.3 Utilizando el diccionario rockyou.txt.....                  | 25 |
| 3.2.4 Opciones para seguir el proyecto .....                      | 28 |
| 3.2.6 Utilizar diccionario personalizado .....                    | 30 |
| 3.2.7 Rompiendo el hash.....                                      | 31 |
| 4. ¿Qué aprendimos con “toodo esto”? .....                        | 33 |
| 4.1 Yescrypt (Ubuntu 22+) .....                                   | 33 |
| yescrypt es intencionalmente difícil de romper. ....              | 33 |
| 4.2 John detecta los yescrypt nuevos, pero hashcat no .....       | 33 |
| 4.3 Para practicar cracking real conviene usar SHA512 .....       | 33 |
| 4.4 ¿John es inútil con una contraseña fuerte alfanumérica? ..... | 34 |
| 4.5 En la realidad, ¿Qué algoritmo se usa hoy? .....              | 35 |
| 4.6 ¿Para usar GPU tengo que instalar John en Windows? .....      | 35 |
| 5. Crear un hash simple en texto para practicar .....             | 37 |

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| 5.1 MD5 y John: .....                                                      | 37 |
| 5.2 SHA1 y hashcat .....                                                   | 38 |
| 5.3 ¿Que aprendimos con esto? .....                                        | 40 |
| 5.3.1 SHA512, SHA256, MD5, SHA1 si funcionan muy bien en John/Hashcat..... | 40 |
| 5.3.4. ¿Qué significa esto para Blue Team? .....                           | 40 |
| 5.4 Tabla comparativa de hashes (dificultad real hoy) .....                | 41 |
| 6. Romper hashes clásicos de Metasploitable2.....                          | 42 |
| 7. Ataque con diccionario especifico .....                                 | 47 |
| 6. Ataque avanzado con Hashcat desde Windows.....                          | 48 |
| Conclusión.....                                                            | 59 |

OZONE

OZONE

## Introducción

En el ámbito de la ciberseguridad y la administración de sistemas, la protección de las credenciales de los usuarios constituye uno de los pilares fundamentales para garantizar la confidencialidad e integridad de la infraestructura tecnológica. Históricamente, los mecanismos de autenticación han dependido de funciones de *hashing* para almacenar contraseñas de forma segura; sin embargo, la evolución constante de la capacidad de cómputo y el desarrollo de metodologías de ataque avanzadas obligan a una actualización permanente de estos algoritmos.

El presente documento detalla el desarrollo del laboratorio **Cracking de Hashes**, cuyo propósito fundamental es estudiar, implementar y evaluar técnicas de extracción y ruptura de *hashes*.

A lo largo del documento, se abordan metodologías críticas de administración y redes, tales como la transferencia segura de archivos mediante el protocolo SCP entre sistemas operativos Ubuntu Server y Kali Linux, así como la identificación y resolución de fallas de conectividad en servicios de red (SSH).

Asimismo, el proyecto analiza la transición tecnológica en los sistemas Linux modernos, contrastando la resistencia de algoritmos de nueva generación como *yescrypt* frente a funciones clásicas y heredadas (*legacy*) como MD5, SHA1 y SHA512crypt. Finalmente, se establece un análisis comparativo cuantitativo respecto al rendimiento de hardware, demostrando empíricamente la superioridad de las tarjetas de video (GPU) sobre las CPU al enfrentarse a auditorías de contraseñas masivas o diccionarios de alta densidad.

## Requisito de operación

Esta guía es la continuación estratégica de los laboratorios:

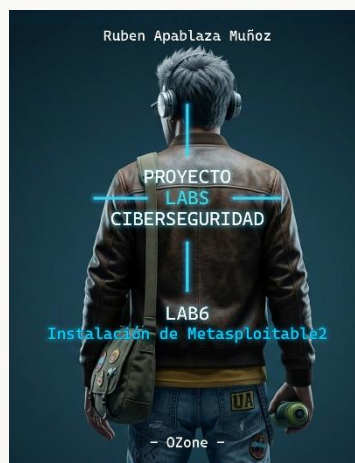
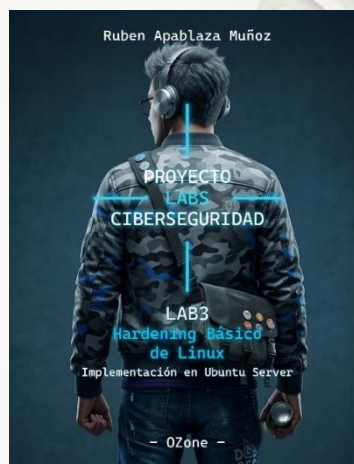
- Lab 1: Instalación y preparación de Ubuntu Server.
- Lab 2: Análisis táctico de logs en sistemas Linux.
- Lab 3: Hardening Básico de Linux.
- Lab 4: Hardening AVANZADO de Linux: UFW + Fail2Ban + CrowdSec + RK Hunter.
- Lab 5: Instalación de Kali Linux
- Lab 6: Instalación de Metasploitable2
- Lab 7: Escaneo de red con Nmap
- Lab 8: Sniffing de red con Wireshark

En ciberseguridad, la integridad del entorno es fundamental. Para garantizar que las acciones que vamos a realizar hoy sean consistentes y que las configuraciones de red no interfieran con tus hallazgos, es **altamente recomendable** haber completado la fase previa.

Si aún no tienes tu entorno listo o quieres asegurarte de que tu máquina cumple con los estándares de este proyecto, puedes realizar ambos laboratorios aquí:



Haz *click* en la imagen para acceder al **Lab** correspondiente.



## **PROYECTO 9: Cracking de Hashes**

### **Objetivo**

- Extraer hashes reales de un sistema controlado
- Entender tipos de hashes comunes
- Romperlos con John the Ripper y Hashcat
- Aprender por que debemos usar contraseñas fuertes y hashing moderno



# 1. Obtener hash real desde Ubuntu Server

En nuestro Ubuntu Server:

## Paso 1: Extraer el hash de un usuario de prueba

Creamos un usuario de laboratorio:

```
sudo adduser prueba
```

Cuando nos pida clave ingresaremos: **cat88**

```
ozone@ubuntuerver:~$ sudo adduser prueba
[sudo] password for ozone:
info: Adding user `prueba' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `prueba' (1001) ...
info: Adding new user `prueba' (1001) with group `prueba (1001)' ...
info: Creating home directory `/home/prueba' ...
info: Copying files from `/etc/skel' ...
New password:
Retype new password:
```

Comprobamos nuestro nuevo usuario creado con

```
cat /etc/passwd | grep prueba
```

```
ozone@ubuntuerver:~$ cat /etc/passwd | grep prueba
prueba:x:1001:1001:,,,:/home/prueba:/bin/bash
ozone@ubuntuerver:~$
```

Luego, extraemos el hash:

```
sudo cat /etc/shadow | grep prueba
```

```
ozone@ubuntuerver:~$ sudo cat /etc/shadow | grep prueba
prueba:$y$j9T$Q./jP37/9P3/xA1gxskdg/$w13oHr4ahGsaf4LGQvyeDsS9eHecl1h0AHigMxnIsv3:20507:0:99999:7:::
ozone@ubuntuerver:~$
```

Ese bloque es un hash **SHA512 shadow**.

Copiamos la línea completa a un archivo llamado:

*hash\_prueba.txt*

para ello agregaremos lo siguiente al comando anterior;

```
sudo cat /etc/shadow | grep prueba > hash_prueba.txt
```

```
ozone@ubuntu:~$ sudo cat /etc/shadow | grep prueba > hash_prueba.txt
ozone@ubuntu:~$ ls
hash_prueba.txt lab
ozone@ubuntu:~$ pwd
/home/ozone
ozone@ubuntu:~$ cat hash_prueba.txt
prueba:$y$j9T$Q./jPJ7/9P3/xA1gxskdg/$w13oHr4ahGsaf4LGQvyeDs59eHecL1h0AHigMXnIsv3:20507:0:99999:7:::
```



## 2. Compartir archivos por SCP

Pasaremos el archivo *hash\_prueba.txt* desde Ubuntu a Kali.

Aprovecharemos esta acción para aprender y revisar un método seguro para compartir y enviar archivos en la red local en este caso entre Ubuntu y Kali.

### METODO SCP (SSH)

Debería funcionar perfecto porque Ubuntu ya tiene SSH, y Kali también.

#### 1. Verificar la IP de Ubuntu

En Ubuntu:

```
ip -4 a
```

Ejemplo:

```
192.168.0.36
```

#### 2. Desde Ubuntu, copiamos el archivo a Kali

Necesitamos la IP de Kali (en modo bridge):

```
ip -4 a
```

Ejemplo:

```
192.168.0.50
```

Ahora... supongamos que en Kali nuestro usuario es “kali”. En Ubuntu ejecutamos:

```
scp hash_prueba.txt kali@192.168.0.50:/home/kali/
```

```
ozone@ubuntuerver:~$ scp hash_prueba.txt ozone@192.168.0.38:/home/ozone/
```



Quizás aparezca lo siguiente:

```
ozone@ubuntuuser:~$ scp hash_prueba.txt ozone@192.168.0.38:/home/ozone
ssh: connect to host 192.168.0.38 port 22: Connection refused
scp: Connection closed
ozone@ubuntuuser:~$
```

Y es por este tipo de cosas que siempre digo que es bueno equivocarse y sobre todo es mejor documentar porque aquí es cuando se aprende, cuando las cosas no nos resultan a la primera.

El mensaje de “conexión rechazada” sucede porque Kali es un sistema pensado (por decirlo de algún modo) para “ataque” y no como un servidor, Por eso:

- el servicio SSH viene **apagado por defecto**.
- no está escuchando en ningún puerto
- no acepta conexiones entrantes

Es totalmente normal y esperado.

Entonces vamos a revisar el **estado** del servicio **ssh** en Kali:

```
sudo systemctl status ssh
```

```
(ozone@kali)-[~]
$ sudo systemctl status ssh
[sudo] password for ozone:
Sorry, try again.
[sudo] password for ozone:
o ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: disabled)
   Active: inactive (dead)
     Docs: man:sshd(8)
           man:sshd_config(5)
```

Y como se puede ver en la imagen, en mi caso el servicio aparece **disabled**, por este motivo no pudo realizarse la conexión SCP. Para solucionar esto, simplemente haremos lo siguiente:

```
sudo systemctl start ssh
```

verificamos con `sudo systemctl status ssh` y comprobamos que ahora si está activo.

```
(ozone@kali)-[~]
$ sudo systemctl start ssh

(ozone@kali)-[~]
$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: disabled)
   Active: active (running) since Mon 2026-02-23 16:44:32 -03; 3s ago
     Invocation: 4be:
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 9605 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 9608 (sshd)
      Tasks: 1 (limit: 9244)
     Memory: 2.4M (peak: 2.8M)
        CPU: 20ms
      CGroup: /system.slice/ssh.service
              └─9608 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Feb 23 16:44:32 kali systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Feb 23 16:44:32 kali sshd[9608]: Server listening on 0.0.0.0 port 22.
Feb 23 16:44:32 kali sshd[9608]: Server listening on :: port 22.
Feb 23 16:44:32 kali systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
```

Si deseamos habilitarlo para que no vuelva a aparecer el error después de reiniciar, hacemos lo siguiente:

```
sudo systemctl enable ssh
```

y podemos ver que el servicio ahora está habilitado

```
(ozone@kali)-[~]
$ sudo systemctl enable ssh
Synchronizing state of ssh.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
Created symlink '/etc/systemd/system/sshd.service' → '/usr/lib/systemd/system/ssh.service'.
Created symlink '/etc/systemd/system/multi-user.target.wants/ssh.service' → '/usr/lib/systemd/system/ssh.service'.

(ozone@kali)-[~]
$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-02-23 16:44:32 -03; 3min 58s ago
     Invocation:
       Docs: man:sshd(8)
             man:sshd_config(5)
    Main PID: 9608 (sshd)
      Tasks: 1 (limit: 9244)
     Memory: 1.9M (peak: 2.8M)
        CPU: 20ms
      CGroup: /system.slice/ssh.service
              └─9608 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Feb 23 16:44:32 kali systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Feb 23 16:44:32 kali sshd[9608]: Server listening on 0.0.0.0 port 22.
Feb 23 16:44:32 kali sshd[9608]: Server listening on :: port 22.
Feb 23 16:44:32 kali systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
```

Ahora podemos intentar nuevamente desde Ubuntu enviar el archivo, en mi caso:

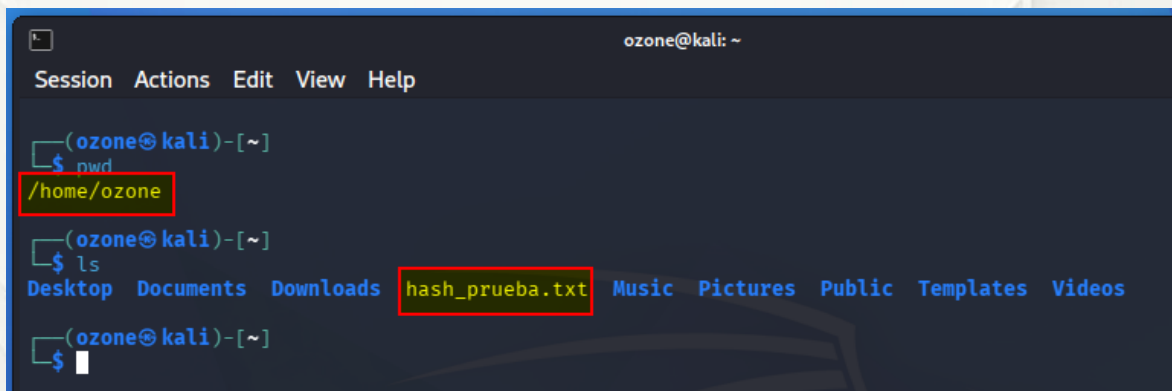
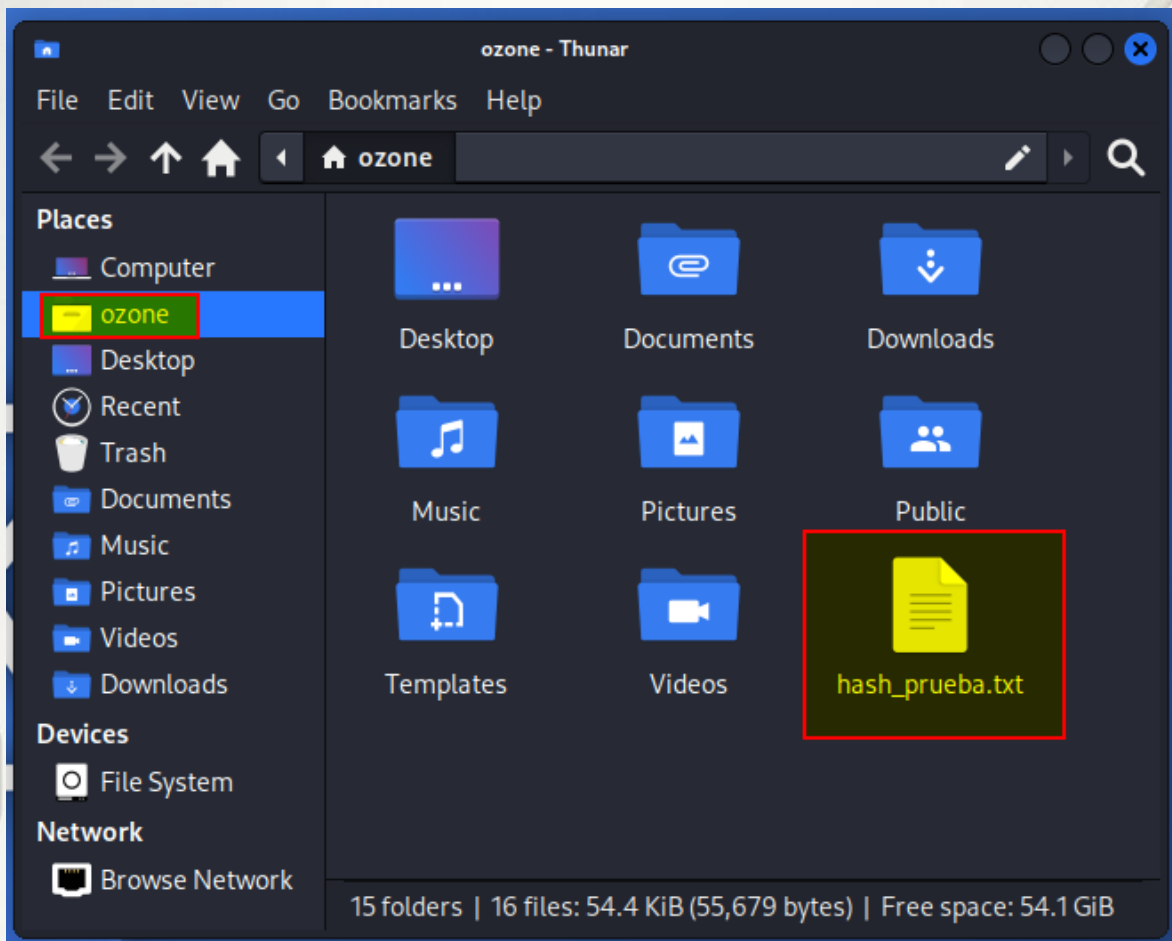
```
scp hash_prueba.txt ozone@192.168.0.38:/home/ozone/
```

```
ozone@ubuntu-server:~$ scp hash_prueba.txt ozone@192.168.0.38:/home/ozone/
```

Y ahora sí, luego de solicitarnos las credenciales y establecer una primera conexión entre ambas maquinas con un hash, el archivo “*hash\_prueba.txt*” es enviado a la maquina Kali.

```
scp: Connection closed
ozone@ubuntu-server:~$ scp hash_prueba.txt ozone@192.168.0.38:/home/ozone
The authenticity of host '192.168.0.38 (192.168.0.38)' can't be established.
ED25519 key fingerprint is SHA256:
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.0.38' (ED25519) to the list of known hosts.
ozone@192.168.0.38's password:
hash_prueba.txt
100% 100 52.3KB/s 00:00
ozone@ubuntu-server:~$
```





En Kali:

Verificamos que nuestro archivo llego correctamente:

```
nano hash_prueba.txt
```

```
ozone@kali: ~  
Session Actions Edit View Help  
GNU nano 8.7 hash_prueba.txt  
prueba:$y$j9T$Q./jPJ7/9P3/xA1gxskdg/$W13oHr4ahGsaf4LGQvyeDsS9eHecL1h0AHigMXnIsv3:20507:0:99999:7:::
```

### \*\*\* ACLARACION.

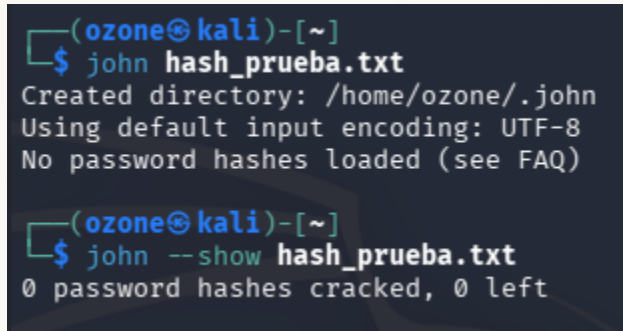
Hubiese sido mucho más simple ya que estoy en un laboratorio con VS Code conectado a Ubuntu por **ssh**, copiar la llave desde Ubuntu (VS Code) **Ctrl+Shift+C** y pegarla directamente en un archivo creado en Kali Linux con **nano** y **Ctrl+Shift+V**, pero creo importante, para quien no lo sabía, aprender a enviar archivos por **SCP** que como podemos ver es una forma que nos permite enviar cualquier tipo de archivo de forma segura y rápida por la red local.



### 3. Romper el hash (Kali)

Ahora que tenemos nuestro archivo en Kali, ejecutamos John:

```
john hash_prueba.txt
```



```
(ozone@kali)-[~]  
$ john hash_prueba.txt  
Created directory: /home/ozone/.john  
Using default input encoding: UTF-8  
No password hashes loaded (see FAQ)  
  
(ozone@kali)-[~]  
$ john --show hash_prueba.txt  
0 password hashes cracked, 0 left
```

Se puede ver en la imagen que John no pudo romper el hash. Esto se debe a que la línea del hash generado NO es un hash SHA512 clásico (\$6\$), es un **hash yescrypt** (formato moderno de Ubuntu 22 y versiones posteriores). Por eso John me dice **“No password hashes loaded”**. John (*en teoría*) la puede romper, pero **necesita que le especifique el formato**.

Antes de intentar descryptar, lo primero sería saber qué tipo de hash es, para saber eso revisaremos lo siguiente.



### 3.1 Confirmar el tipo de hash

En la página oficial de la documentación de Ubuntu, nos señala que desde la versión 22, el algoritmo de hashing es “**yescrypt**”.

Ubuntu security documentation

Go

Search

Software integrity

Security updates

Ubuntu security features

Overview of security features

Platform protections

Privilege restriction

Cryptography

Cloud PRNG seed

Disable legacy TLS

Affected libraries and configuration changes

Legacy TLS re-enablement (not recommended)

Common issues and troubleshooting

Security implications and best practices

Additional resources

Password hashing

Cryptographic algorithms

Process and memory protections

Give feedback

🌙

## Password hashing

Cryptographic hashing allows you to store passwords without revealing their contents. You can think of the hash as a digital fingerprint. It has an insignificantly low probability of two different passwords sharing the same fingerprint, while making it computationally intractable to recover the plain text password from the fingerprint. When verifying a password, the system computes its hash and compares it to the stored hash.

The system adds a cryptographic salt to passwords before hashing them to prevent an adversary from precomputing hashes for common passwords. This ensures that the same password used for different logins results in different hashes.

Ubuntu stores password hashes for local users in `/etc/shadow`. See the [shadow\(5\)](#) and [crypt\(5\)](#) manual pages for more information.

| RELEASE                            | HASHING ALGORITHM |
|------------------------------------|-------------------|
| Ubuntu 14.04 LTS (Trusty Tahr)     | SHA-512           |
| Ubuntu 16.04 LTS (Xenial Xerus)    | SHA-512           |
| Ubuntu 18.04 LTS (Bionic Beaver)   | SHA-512           |
| Ubuntu 20.04 LTS (Focal Fossa)     | SHA-512           |
| Ubuntu 22.04 LTS (Jammy Jellyfish) | yescrypt          |
| Ubuntu 24.04 LTS (Noble Numbat)    | yescrypt          |
| Ubuntu 25.04 (Plucky Puffin)       | yescrypt          |
| Ubuntu 25.10 (Quaking Quokka)      | yescrypt          |

Ubuntu 14.04 LTS (Trusty Tahr) through Ubuntu 20.04 LTS (Focal Fossa) use the `SHA-512` hash function, which produces a fixed-length 512-bit output. Although efficient to compute, its usage has declined because brute-forcing is more feasible compared to stronger algorithms.

Ubuntu 22.04 LTS (Jammy Jellyfish) and later use `yescrypt`, which is based on `scrypt`, a computationally expensive key derivation function. This makes it less practical for adversaries to

En este punto alguien podría pensar que hay herramientas que pueden ayudar a identificar el tipo de hash y probablemente tengan razón. Pero como siempre digo, probemos:

1. Una opción antigua es utilizar una herramienta que viene en Kali, hablo de **hashid**. Lamentablemente el formato **yescript** es muy nuevo para la herramienta y no puede ser reconocido como muestra la imagen.

```
(ozone@kali)-[~]  
$ hashid -e "$y$j9T$Q./jPJ7/9P3/xA1gxskdg/$W13oHr4ahGsaf4LGQvyeDsS9eHecL1h0AHigMXnIsv3~"  
Analyzing './jPJ7/9P3/xA1gxskdg/~'  
[+] Unknown hash
```

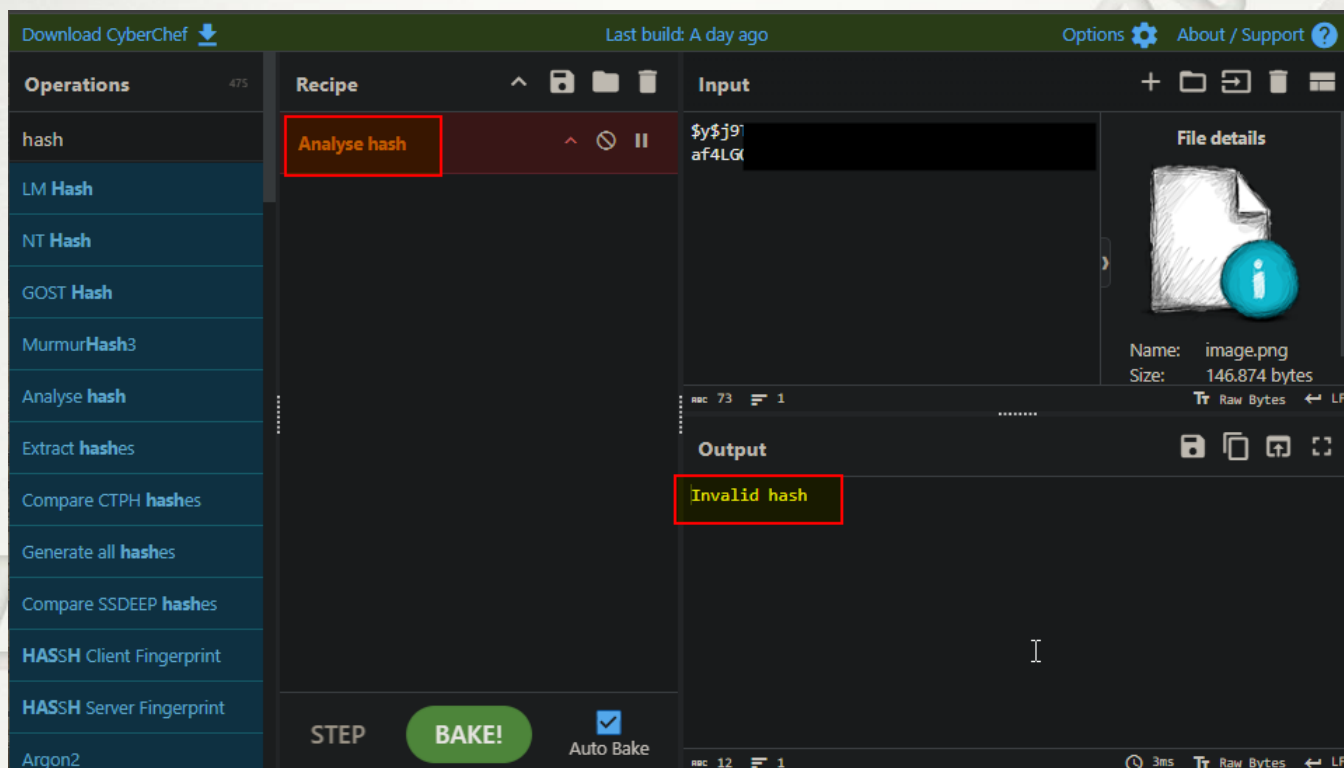
2. Otra opción válida es utilizar el mismo John the Ripper, así que le vamos a preguntar a John si es que es capaz de reconocer el formato yescript. Esto además nos servirá para identificar si **John the Ripper** puede hacer el trabajo de romper el hash. Y como podemos ver en la imagen, la verdad es que la versión que tengo en mi Kali, no reconoce el formato yescript.

```
(ozone@kali)-[~]  
$ john --list=formats | grep -i yes  
416 formats (149 dynamic formats shown as just "dynamic_n" here)
```

3. Una siguiente opción sería utilizar **hashcat**. Y del mismo modo que hice con John le pregunto a hashcat. Lamentablemente la respuesta es la misma que la de John. Este ni siquiera tiene la cortesía de responder.

```
(ozone@kali)-[~]  
$ hashcat --help | grep -i yescript  
  
(ozone@kali)-[~]
```

4. Buscaremos en el mágico CyberChef (<https://gchq.github.io/CyberChef/>) pero lamentablemente tampoco nos va bien.



5. Como no me quería quedar picado decidí ir por una solución más radical... naaaa, no es para tanto. Hay otra herramienta que debemos instalar en Kali Linux, **name-that-hash**.

Cuando quise instalar me encontré con el siguiente error.

```
(ozone@kali)-[~]
$ pip install name-that-hash
error: externally-managed-environment

× This environment is externally managed
╰─> To install Python packages system-wide, try apt install
python3-xyz, where xyz is the package you are trying to
install.

If you wish to install a non-Kali-packaged Python package,
create a virtual environment using python3 -m venv path/to/venv.
Then use path/to/venv/bin/python and path/to/venv/bin/pip. Make
sure you have pypy3-venv installed.

If you wish to install a non-Kali-packaged Python application,
it may be easiest to use pipx install xyz, which will manage a
virtual environment for you. Make sure you have pipx installed.

For more information, refer to the following:
* https://www.kali.org/docs/general-use/python3-external-packages/
* /usr/share/doc/python3.13/README.venv

note: If you believe this is a mistake, please contact your Python installation
your Python installation or OS, by passing --break-system-packages.
hint: See PEP 668 for the detailed specification.
```

Si bien ahí me da la solución para instalar, lo cierto es que eso de “romper el sistema de paquetes” no me gustó mucho, así que le di la vuelta como nuestro a continuación.

Para instalar **name-that-hash**, primero debemos instalar lo siguiente. “**pipx**” ¿Y que es pipx?

“pipx en Kali Linux es una herramienta diseñada para instalar y ejecutar aplicaciones de Python de línea de comandos en entornos virtuales aislados. A diferencia de pip tradicional, pipx mantiene las dependencias de cada herramienta separadas, evitando conflictos con las librerías del sistema y asegurando que las herramientas de seguridad estén siempre disponibles de manera global.” (<https://github.com/pypa/pipx>).



Básicamente sirve para instalar sin romper nada por lo de los entornos aislados.

Así que vamos a instalar ejecutando y siguiendo el siguiente orden:

- `sudo apt install pipx`
- `pipx ensurepath`
- `pipx install name-that-hash`

```
(ozone@kali)-[~]
$ sudo apt install pipx
[sudo] password for ozone:
pipx is already the newest version (1.8.0-1).
pipx set to manually installed.
The following packages were automatically installed and are no longer required:
  bloodhound.py  libavformat61  libmjpegutils-2.1-0t64  libpocketsphinx3  libvdpau-v4l2  python3-aiomcache  ruby-unf-ext
  curlftpfs      libconfig-inifiles-perl  libmpeg2encpp-2.1-0t64  libpostproc58     mesa-vdpau-drivers  python3-fs         vdpau-driver-all
  libaudio2      libfuse2t64      libmpeg2-2.1-0t64      libsphinxbase3t64  pocketsphinx-en-us  python3-wapiti-arsenic  python3-yaswfp
  libavfilter10  libgav1-1        libmupdf25.1           libswscale8        python3-aiocache    python3-yaswfp
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 119

(ozone@kali)-[~]
$ pipx ensurepath
/home/ozone/.local/bin is already in PATH.

▲ All pipx binary directories have been appended to PATH. If you are sure you want to proceed, try again with the '--force' flag.

Otherwise pipx is ready to go! 🌟 🌟 🌟

(ozone@kali)-[~]
$ pipx install name-that-hash
installed package name-that-hash 1.11.0, installed using Python 3.13.11
These apps are now globally available
- name-that-hash
- nth
done! 🌟 🌟 🌟
```

Una vez instalado, ejecutamos

`nth --text 'hash' (fijarse en las comillas simples al inicio y al final).`

```
(ozone@kali)-[~]
$ nth --text 'v3'
```

```
(ozone@kali)-[~]
$ nth --text 'v3'

Name-That-Hash

https://twitter.com/bee_sec_san
https://github.com/HashPals/Name-That-Hash

$y$j v3

Most Likely
yescrypt, HC: Not yet supported, see notes in summary. JtR: On systems that use libxcrypt, y
uses the system's crypt function. Summary: Can be used in Linux Shadow Files in modern Linux
hashcat this is not yet implemented, please vote (👍 "thumbs up") on this issue: https://git
```

formato yescript

## 3.2. Intentar romper el hash con formato yescript

### 3.2.1 Opción John the Ripper

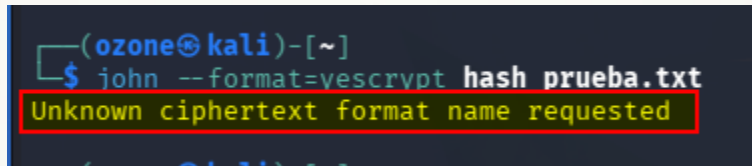
Ya sabemos a esta altura que **mi** Kali **PARECE** (nótese el “parece” y dejémoslo en standby para más adelante) que no puede descryptar yescript, pero si pudiera esto es lo que tendría que hacer en primera instancia indicándole el formato.

```
john --format=yescript hash_prueba.txt
```

Cuando termine:

```
john --show hash_prueba.txt
```

Pero como ya dije, esta es la respuesta negativa que obtengo en **mi** caso.



```
(ozone@kali)-[~]  
$ john --format=yescript hash prueba.txt  
Unknown ciphertext format name requested
```

Como ya había mostrado previamente, mi versión de John the Ripper en Kali NO trae soporte para yescript compilado. Por eso me dice:

*Unknown ciphertext format name requested*

Es normal, varias builds de John en Kali no incluyen ese formato.

En este punto vamos a darnos una vuelta que es necesaria para entender:

### 3.2.2 Opción simple: Hashcat

Hashcat si soporta yescrypt (pero con limitaciones que afloran mas adelante y por eso la importancia de documentar estos procesos).

En Kali:

#### 1. Ver si nuestra GPU soporta hashcat:

hashcat -I

```
(ozone@kali)-[~]
$ hashcat -I
hashcat (v7.1.2) starting in backend information mode

OpenCL Info:
=====

OpenCL Platform ID #1
Vendor..: The pocl project
Name....: Portable Computing Language
Version.: OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_D
EBUG

Backend Device ID #01
Type.....: CPU
Vendor.ID....: 128
Vendor.....: GenuineIntel
Name.....: cpu-haswell-Intel(R) Core(TM) i9-10900KF CPU @ 3.70GHz
Version.....: OpenCL 3.0 PoCL HSTR: cpu-x86_64-pc-linux-gnu-haswell
Processor(s) ...: 4
Preferred.Thrd.: 1
Clock.....: 3696
Memory.Total...: 5861 MB (limited to 1024 MB allocatable in one block)
Memory.Free....: 2930 MB
Memory.Unified.: 1
Local.Memory...: 512 KB
OpenCL.Version.: OpenCL C 1.2 PoCL
Driver.Version.: 6.0+debian
```

En este caso nos encontramos que Kali al estar montado sobre una VM en un sistema Windows no detecta la GPU y solo detecta la CPU del equipo anfitrión de la VM, dicho de otra forma, la capa de abstracción (ese concepto que aquí cobra vida), VMWare es un hipervisor que **emula** un hardware genérico; aunque tengamos una “Nvidia rtx 5900 supersaiyayin nivel dios”, la VM solo ve un adaptador de pantalla básico; en otras palabras, una VM no puede “hablar” directamente con el hardware de la maquina anfitriona.

Si queremos utilizar la **GPU** con **Hashcat** tenemos las siguientes opciones:

| Opción                                   | Dificultad | Recomendación                           | Estado real de GPU                                             |
|------------------------------------------|------------|-----------------------------------------|----------------------------------------------------------------|
| Instalar hashcat en Windows              | Muy fácil  | La mejor opción                         | GPU funcional al 100% sin configurar nada                      |
| Dual Boot (Kali instalado en disco)      | Media      | Buen equilibrio                         | GPU soportada si instalas drivers Nvidia/AMD                   |
| GPU Passthrough (VM con PCI Passthrough) | Muy alta   | Solo si sabemos virtualización avanzada | GPU funcional, pero complejo y delicado                        |
| Kali Live (USB)                          | Fácil      | No recomendado para hashcat con GPU     | GPU NO funcional por defecto (sin drivers, sin kernel modules) |

De todos modos seguiremos con hashcat y CPU

**2. Yescript es el modo = 1800** *\*\*esto es un error y más adelante veremos por qué.*

```
hashcat -m 1800 hash_prueba.txt rockyou.txt --force
```

Si nuestro Kali no tiene GPU, usa CPU igual con **--force**, hashcat va a tardar más, pero funciona.

```
(ozone@kali)-[~]
$ hashcat -m 1800 hash_prueba.txt rockyou.txt --force
hashcat (v7.1.2) starting

You have enabled --force to bypass dangerous warnings and errors!
This can hide serious problems and should only be done when debugging.
Do not report hashcat issues encountered when using --force.

rockyou.txt: No such file or directory

Started: Tue Feb 24 17:46:40 2026
Stopped: Tue Feb 24 17:46:41 2026
```

Ahora vemos que nos encontramos con otro “pequeño” inconveniente.

Básicamente, Hashcat me está diciendo: *"Entiendo lo que quieres hacer, pero no encuentro el diccionario de contraseñas"*.



### 3.2.3 Utilizando el diccionario rockyou.txt

En Kali Linux, el archivo **rockyou.txt** (diccionario de contraseñas) no está "suelto" en la carpeta donde estamos trabajando; se encuentra en un directorio específico del sistema y, por defecto, viene comprimido para ahorrar espacio.

Aquí vemos los pasos exactos para solucionarlo:

#### 1. Descomprimir rockyou (si no lo hemos hecho)

Por defecto, Kali guarda este diccionario en `/usr/share/wordlists/`. Si es la primera vez que lo usamos, estará en formato **.gz**

```
sudo gzip -d /usr/share/wordlists/rockyou.txt.gz
```

```
(ozone@kali)-[~]  
$ sudo gzip -d /usr/share/wordlists/rockyou.txt.gz  
gzip: /usr/share/wordlists/rockyou.txt.gz: No such file or directory
```

Y podemos ver que el archivo sigue sin ser encontrado porque no está instalado, así que procedemos a ello:

```
sudo apt update && sudo apt install wordlists -y
```

```
(ozone@kali)-[~]  
$ sudo apt update && sudo apt install wordlists -y  
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]  
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [20.6 MB]  
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [52.3 MB]  
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [116 kB]  
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [273 kB]  
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [184 kB]  
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [877 kB]  
Get:8 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [11.8 kB]  
Fetched 74.4 MB in 7s (10.9 MB/s)  
119 packages can be upgraded. Run 'apt list --upgradable' to see them.  
wordlists is already the newest version (2026.1.2).  
wordlists set to manually installed.  
The following packages were automatically installed and are no longer required:  
bloodhound.py libgav1-1 libsphinxbase3t64 python3-fs  
curlftpfs libjpegutils-2.1-0t64 libswscale8 python3-wapiti-arsenic  
libaudio2 libmpeg2encpp-2.1-0t64 libvdpau-va-gl1 python3-yaswfp  
libavfilter10 libmpeg2-2.1-0t64 mesa-vdpau-drivers ruby-unf-ext  
libavformat61 libmupdf25.1 pocketsphinx-en-us vdpau-driver-all  
libconfig-inifiles-perl libpocketsphinx3 python3-aiocache  
libfuse2t64 libpostproc58 python3-aiomcache  
Use 'sudo apt autoremove' to remove them.  
  
Summary:  
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 119
```

0Z

esto es normal, es  
o fue instalado con  
oad/data/rockyou.1

```

(ozone@kali)~[~]
$ wget https://github.com/brannondorsey/naive-hashcat/releases/download/data/rockyou.txt
--2026-02-24 18:16:47-- https://github.com/brannondorsey/naive-hashcat/releases/download/data/rockyou.txt
Resolving github.com (github.com)... 4.228.31.150
Connecting to github.com (github.com)|4.228.31.150|:443 ... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://release-assets.githubusercontent.com/github-production-release-asset/97553311/d4f580f8-6b49-
1e7-8f70-7f460f85ab3a?sp=r&sv=2018-11-09&sr=b&spr=https&se=2026-02-24T21%3A58%3A41Z&srcd=attachment%3B+filename%
3Drockyou.txt&trst=application%2Foctet-stream&skoid=96c2d410-5711-43a1-aedd-ab1947aa7ab0&sktid=398a6654-997b-b-
7e9b-126-951b896b4de&skt=2026-02-24T20%3A58%3A39Z&ske=2026-02-24T21%3A58%3A41Z&skv=2018-11-09&sig=mwqavi-
YbNuQpvPeIsbW5mWcEbDZshZuEbg08%2F0WRA%2Fg%3D6jwt=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJnaXRodWUuY29t
iWlYXVkJ0iJoicVsZWZfZS1hc3NldHMUz2LoaHVIDXNlcmVubnRlbnQvY291Iiwia2V5Ijoia2V5MSIsImV4CiC1MTk3MTk3MTQwYmVwb3MjIjo
Nzc0OTY3ODM0ZCJwYXN0IjoicmVsZWZfZWZfc2V0cHJvZHVjdGlib3V5bG9iLmNvcmdUd2luZ3R5cy5XQXQifQ.3cyi26e2jSf4B8oM-a5s0A-
v0SnnrvU5CU3AByzAQ06response-content-disposition=attachment%3B%20filename%3Drockyou.txt&response-content-type=

```

0Z

0Z

```
(ozone@kali)-[~]
$ ls -l rockyou.txt
-rw-rw-r-- 1 ozone ozone 139921497 Dec  8  2021 rockyou.txt

(ozone@kali)-[~]
$ ls
Desktop Downloads hash_prueba.txt Music Public Templates
Documents hash_prueba2.txt mini.txt Pictures rockyou.txt Videos
```

Y

```
sudo mv rockyou.txt /usr/share/wordlists/
```

```
(ozone@kali)-[~]  
$ sudo mv rockyou.txt /usr/share/wordlists
```

Y

```
ls -l /usr/share/wordlists | grep rockyou.txt
```

```
(ozone@kali)-[~]
$ ls -l /usr/share/wordlists | grep rockyou.txt
-rw-rw-r-- 1 ozone ozone 139921497 Dec  8 2021 rockyou.txt
```

Ahora ya podemos lanzar nuestro ataque con **hashcat**

```
hashcat -m 1800 hash_prueba.txt /usr/share/wordlists/rockyou.txt
```

```
(ozone@kali)-[~]
$ hashcat -m 1800 hash_prueba.txt /usr/share/wordlists/rockyou.txt
hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEP, DISTRO, POCL_DE
BUG) - Platform #1 [The pocl project]

=====
* Device #01: cpu-haswell-Intel(R) Core(TM) i9-10900KF CPU @ 3.70GHz, 2930/5861 MB (1024 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256
Minimum salt length supported by kernel: 0
Maximum salt length supported by kernel: 256

Hashfile 'hash_prueba.txt' on line 1 (prueba...h0AHigMXnIsv3:20507:0:99999:7::): Token length exception

* Token length exception: 1/1 hashes
  This error happens if the wrong hash type is specified, if the hashes are
  malformed, or if input is otherwise not as expected (for example, if the
  --username or --dynamic-x option is used but no username or dynamic-tag is present)

No hashes loaded.

Started: Tue Feb 24 18:45:15 2026
Stopped: Tue Feb 24 18:45:15 2026
```

Y nos informa que no se cargaron hashes. Esto es porque nuestro **hash NO es yescrypt tradicional -m 1800** (por eso en el punto 3.2.2 puse que esto es un error).

**Ubuntu 24.04** usa yescrypt con variante **yescrypt-default**, y hashcat **NO lo soporta aun completamente**. Por eso nos lanza:

*Token length exception*

No es el comando, tampoco es el archivo, es limitación de hashcat.

**¿Como se confirma?**

El hash empieza con:

**\$y\$j9T\$**

El yescrypt **clásico** que hashcat soporta (modo 1800) usa:

**\$y\$...**

Pero la variante nueva (**\$y\$j9T\$**) no funciona en **hashcat 7.1.2**.



### 3.2.4 Opciones para seguir el proyecto

Volver a John the Ripper (*solo john soporta yescrypt moderno*). John soporta **yescrypt moderno** si lo usamos así:

```
john --format=crypt --wordlist=/usr/share/wordlists/rockyou.txt hash_prueba.txt
```

```
(ozone@kali)-[~]
$ john --format=crypt --wordlist=/usr/share/wordlists/rockyou.txt hash_prueba.txt
Using default input encoding: UTF-8
Loaded 1 password hash (crypt, generic crypt(3) [?/64])
Cost 1 (algorithm [1:descrypt 2:md5crypt 3:sunmd5 4:bcrypt 5:sha256crypt 6:sha512crypt]) is 0 for all loaded hashes
Cost 2 (algorithm specific iterations) is 1 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:06 0.01% (ETA: 11:09:34) 0g/s 275.1p/s 275.1c/s 275.1C/s 2hot4u..eastside
█
```

Y vemos que **ya funcionó**. John reconoció el hash yescrypt moderno como **crypt genérico** y ahora **lo está rompiendo**.

Que estamos viendo:

*Press 'q' or Ctrl-C to abort, almost any other key for status*

Significa que el cracking está en progreso.

#### ¿Como ver el progreso?

Presionamos cualquier tecla (no Enter) en esa misma terminal.



### 3.2.5 La demora para efecto de las pruebas de laboratorio.

Esto no es un problema de la herramienta ni nada, pero como estoy haciendo un lab no dejare que se demore horas de horas intentando crackear la contraseña.

La clave que puse fue **cat88** que es una clave bastante fácil de romper.

Mientras espero me surgen las siguientes dudas:

#### *¿Existe algún modo de acelerarlo con el uso de la gpu o ya la está utilizando?*

Como dijimos previamente, ¿John está usando GPU?

**No.**

Recordemos que John The Ripper en Kali, por defecto, usa CPU, incluso si tenemos GPU en el host. En una **VM**, la GPU no se pasa a la máquina virtual, así que ninguna VM (ni Kali ni Ubuntu) puede usar la GPU física salvo que configuremos **PCI Passthrough**, cosa compleja y no necesaria para este laboratorio. Por lo tanto no hay forma sencilla de usar la GPU dentro de una VM normal, John está usando solo CPU y por eso es más lento.

#### *¿Por qué yescrypt es tan lento?*

Yescrypt moderno (Ubuntu 22-24) está diseñado para ser:

- muy resistente a GPU
- muy lento incluso en CPU
- muy costoso de romper

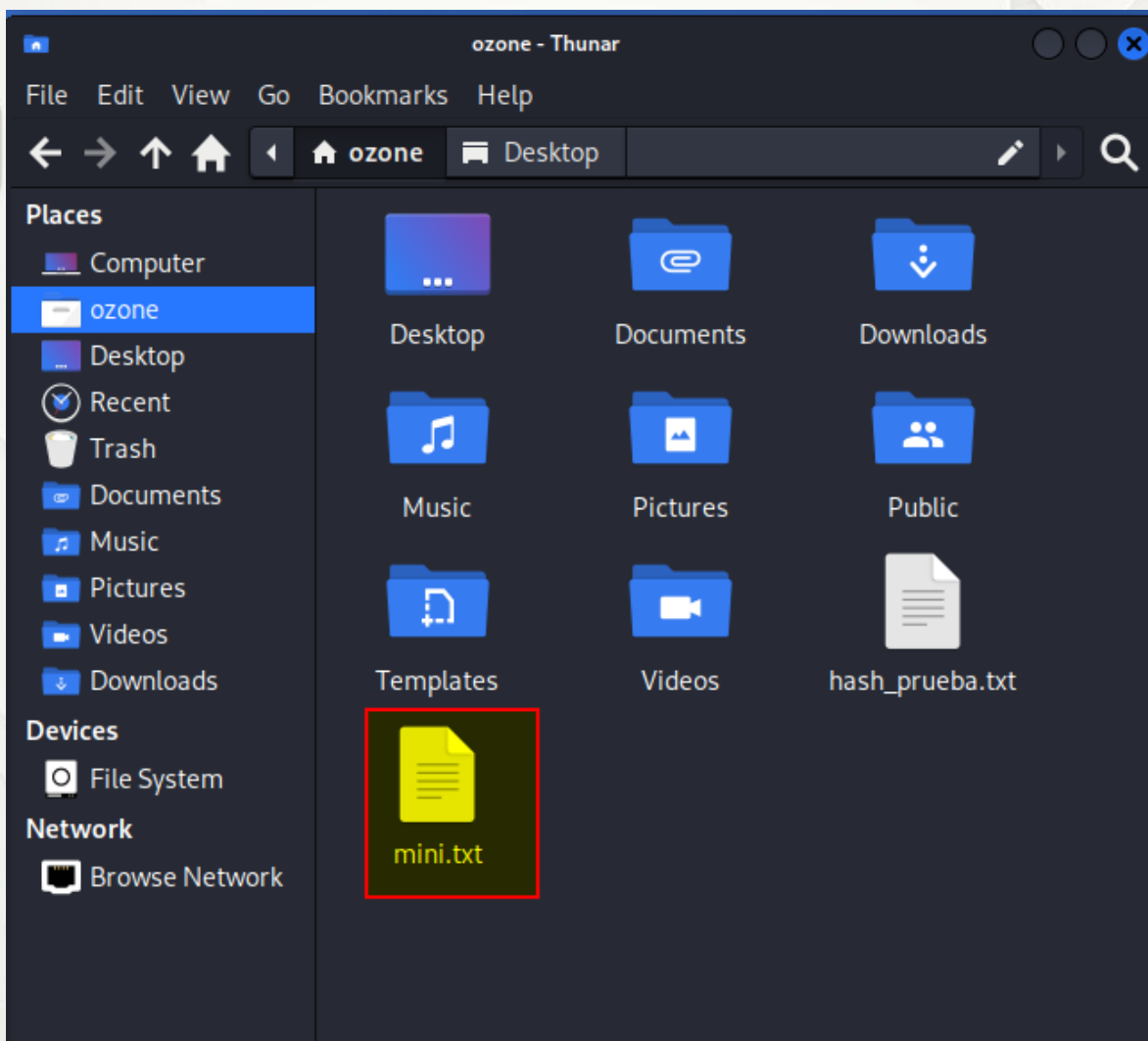
Por eso **cat88** va a tardar, pero debería caer eventualmente porque no es una clave fuerte.

### 3.2.6 Utilizar diccionario personalizado

Para efectos de acelerar el resultado **solo para este lab** (esto podría realizarse en un escenario real haciendo un muy buen reconocimiento de la víctima (OSINT), rellenando un diccionario con palabras “clave” como su dirección, nombre de mascota y toda la info que nos pueda dar pistas de posibles palabras clave y que la misma tenga una clave tan débil como la que estamos utilizando) crearé un **mini-diccionario** con la clave incluida para que John the Ripper busque en dicho archivo.

Creamos el mini-diccionario:

```
echo cat88 > mini.txt
```



### 3.2.7 Rompiendo el hash.

Y ahora rompemos el hash con **John the Ripper** finalmente:

```
john --format=crypt --wordlist=mini.txt hash_prueba.txt
```

```
(ozone@kali)-[~]
$ john --format=crypt --wordlist=mini.txt hash_prueba.txt
Using default input encoding: UTF-8
Loaded 1 password hash (crypt, generic crypt(3) [?/64])
Cost 1 (algorithm [1:descrypt 2:md5crypt 3:sunmd5 4:bcrypt 5:sha256crypt 6:sha512crypt]) is 0 for all loaded hashes
Cost 2 (algorithm specific iterations) is 1 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Warning: Only 1 candidate left, minimum 96 needed for performance.
cat88 (prueba)
ig 0:00:00:00 DONE (2020-02-24 18:57) 50.00g/s 50.00p/s 50.00c/s 50.00C/s cat88
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

Funcionó exactamente como debía. John rompió el hash **yescrypt** correctamente:

**cat88**

Y lo hizo al instante porque usamos un diccionario mínimo. La advertencia que nos muestra:

*Warning: Only 1 candidate left, minimum 96 needed for performance.*

solo indica que John no aprovecha todo su rendimiento con tan pocas palabras, pero igual funciona.

Si queremos ver la contraseña extraída:

```
john --show hash_prueba.txt
```

```
(ozone@kali)-[~]
$ john --show hash_prueba.txt
prueba:cat88 20507:0:99999:7 :::
1 password hash cracked, 0 left
```

John the Ripper guarda las contraseñas extraídas en:

`/home/ozone/.john/john.pot`

Por lo que si queremos revisarlas:

```
cat /home/ozone/.john/john.pot
```

```
(ozone@kali)-[~]
$ cat /home/ozone/.john/john.pot
$y$j9T$Q../jPj7/9P3/xA1gxskdg/$W13oHr4ahGsaf4LGQvyeDsS9eHecL1h0AHigMXnIsv3 cat88
```

Esto confirma 3 cosas importantes del Laboratorio 9:

- El hash estaba bien extraído.
- El formato de John era el correcto (*--format=crypt*).
- La contraseña real se puede recuperar sin problemas.





## 4. ¿Qué aprendimos con “toodo esto”?

### 4.1 Yescript (Ubuntu 22+)

**yescript es intencionalmente difícil de romper.** Las versiones nuevas de Ubuntu usan yescript justamente para esto:

- que John no pueda romperlo fácilmente
- que hashcat no pueda usar GPU
- que el cracking sea ineficiente y lento incluso con hardware moderno
- que diccionarios gigantes no sirvan si la clave es decente

yescript está diseñado para ser:

- memory-hard (usa mucha RAM)
- CPU-hard (penaliza procesamiento)
- anti-GPU

Por eso, en el laboratorio:

- John no lo detecto bien
- Hashcat no lo soporta
- Solo funcionó un ataque con `--format=crypt` + diccionario
- Solo funcionó porque la clave era débil (“**cat88**”).

Si la clave fuera fuerte:

- ni John ni Hashcat lo romperían.
- tardaríamos semanas o meses.
- en VM con CPU sería imposible

### 4.2 John detecta los yescript nuevos, pero hashcat no

En máquinas modernas, John suele ser mejor para yescript.

### 4.3 Para practicar cracking real conviene usar SHA512

Porque:

- Es más común en sistemas legacy

- Es soportado por hashcat
- Se puede acelerar con GPU si no estas en VM

#### 4.4 ¿John es inútil con una contraseña fuerte alfanumérica?

No es inútil. Pero no rompe una contraseña fuerte moderna si usa un hash muy resistente como yescrypt.

Ejemplo:

- Contraseña: g4Kny3A26#Chlle
- Hash: yescrypt (Ubuntu 22+)

Resultado realista:

Ni John ni Hashcat la rompen por fuerza bruta o diccionario. Tardaría siglos, incluso con GPU. Eso es justo lo que se espera de un buen hash moderno. Pero John **NO** es inútil. Es excelente en:

- SHA512crypt
- MD5crypt
- Bcrypt
- NTLM
- LM
- ZIP/RAR
- PDF
- SSH private keys
- Kerberos tickets
- Web passwords con formatos comunes

En hashes modernos como yescrypt o Argon2, simplemente la seguridad es demasiado alta.

*Nota:* Aunque una contraseña sea larga, si el algoritmo es rápido (como MD5), un clúster de GPUs podría probar billones de combinaciones. Pero con **Yescrypt**, debido a que usa mucha RAM por cada intento, el costo económico y de tiempo para un atacante es simplemente demasiado alto.

## 4.5 En la realidad, ¿Qué algoritmo se usa hoy?

- Linux:
  - Ubuntu 22/23/24 > yescrypt (actual)
  - Debian 12 > yescrypt
  - Arch > yescrypt
  - RHEL/Rocky/Alma > SHA512crypt
- Windows:
  - NTLM (muy débil)

Por eso se ve tanto crackeo en entornos empresariales.

- Navegadores / Web:  
bcrypt / Argon2 / scrypt (muy seguros)

## 4.6 ¿Para usar GPU tengo que instalar John en Windows?

SI. En una **VM** (Kali o Ubuntu), la GPU no pasa directamente a la máquina virtual, entonces:

- Hashcat en VM → CPU
- John en VM → CPU

Para usar GPU necesitamos:

### **OPCION A: Instalar Hashcat en Windows**

Es lo más fácil. Detecta mi GPU Nvidia o AMD automáticamente.

### **OPCION B: Instalar John Jumbo compilado con OpenCL**

Funciona, pero es más complejo que hashcat.

### **OPCION C: Pasar GPU a VM (PCI passthrough)**

Se puede solo con:

- Proxmox
- KVM
- QEMU
- ESXi

NO funciona en VirtualBox ni VMware Workstation.



## 5. Crear un hash simple en texto para practicar

En Kali podemos generar un hash Linux sin usar Ubuntu:

### 5.1 MD5 y John:

```
echo -n "123456" | md5sum
```

```
(ozone@kali)-[~]  
$ echo -n "12345" | md5sum  
827ccb0eea8a706c4c34a16891f84e7b -
```

Creamos un archivo:

```
nano md5.txt
```

```
(ozone@kali)-[~]  
$ nano md5.txt
```

Pegamos el hash generado y guarda los cambios

```
ozone@kali: ~  
Session Actions Edit View Help  
GNU nano 8.7 md5.txt *  
827ccb0eea8a706c4c34a16891f84e7b
```

Y crackeamos:

```
john md5.txt --format=raw-md5
```

```
(ozone@kali)-[~]  
$ john md5.txt --format=raw-md5  
Using default input encoding: UTF-8  
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])  
Warning: no OpenMP support for this hash type, consider --fork=4  
Proceeding with single, rules:Single  
Press 'q' or Ctrl-C to abort, almost any other key for status  
Almost done: Processing the remaining buffered candidate passwords, if any.  
Proceeding with wordlist:/usr/share/john/password.lst  
12345 (?)  
1g 0:00:00:00 DONE 2/3 (2026-02-25 16:04) 100.0g/s 38400p/s 38400c/s 38400C/s 123456..larry  
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably  
Session completed.
```

## 5.2 SHA1 y hashcat

En esta ocasión crearemos inmediatamente el archivo **sha1.txt** agregando un **| (pipe)**, quedando de la siguiente forma

```
echo -n "hola123" | sha1sum | cut -d " " -f1 > sha1.txt
```

```
(ozone@kali)-[~]  
$ echo -n "hola123" | sha1sum | cut -d " " -f1 > sha1.txt
```

### Explicación:

- `echo -n "hola123"` genera la clave
- `sha1sum` la convierte a SHA1
- `cut -d " " -f1` toma solo el hash y NO el guion
- `sha1.txt` lleva la salida a un archivo de texto llamado sha1.txt

```
(ozone@kali)-[~]  
$ cat sha1.txt  
0f3fde0103dd44077c040215a2fabd09a097aecc
```

Crackeamos SHA1 con Hashcat

Hashcat usa el modo:

**100 = raw-SHA1**

Ejecutamos:

```
hashcat -m 100 sha1.txt /usr/share/wordlists/rockyou.txt
```

Hashcat detectará CPU debido a que estamos en VM, eso es normal.

```

(ozone@kali)-[~]
$ hashcat -m 100 sha1.txt /usr/share/wordlists/rockyou.txt
hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEP, DISTRO, POCL_D
EBUG) - Platform #1 [The pocl project]

=====
* Device #01: cpu-haswell-Intel(R) Core(TM) i9-10900KF CPU @ 3.70GHz, 2930/5861 MB (1024 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Hash
* Single-Salt
* Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

Host memory allocated for this attack: 513 MB (6214 MB free)

Dictionary cache built:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344391
* Bytes.....: 139921497
* Keyspace...: 14344384
* Runtime...: 1 sec

0f3fde0103dd44077c040215a2fabd09a097aecc hola123

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 100 (SHA1)

```

Vemos el resultado con:

```
hashcat -m 100 sha1.txt --show
```

```

(ozone@kali)-[~]
$ hashcat -m 100 sha1.txt --show
0f3fde0103dd44077c040215a2fabd09a097aecc hola123

```

### 5.3 ¿Que aprendimos con esto?

Con esto se comprueba en relación al caso anterior donde john y hashcat tuvieron dificultades por **yescrypt** que las herramientas funcionan pero que los métodos nuevos como yescrypt aun no pueden ser fácilmente abordados y rotos por john the ripper y hashcat.

#### 5.3.1 SHA512, SHA256, MD5, SHA1 si funcionan muy bien en John/Hashcat

Porque son:

- funciones de hashing, rápidas
- no memory-hard
- tienen kernels compatibles con GPU
- Hashcat puede procesar millones por segundo

Por eso rompimos MD5 y SHA1 sin problema.

#### 5.3.4. ¿Qué significa esto para Blue Team?

- Si un sistema moderno usa yescrypt o Argon2 crackear es prácticamente imposible con hardware común.
- Si un atacante roba */etc/shadow* con yescrypt, no le sirve de mucho si las claves son buenas.
- Si una app usa MD5, SHA1, SHA256 para contraseñas *Vulnerabilidad seria* (crackeas en minutos).



## 5.4 Tabla comparativa de hashes (dificultad real hoy)

| Hash / Método | Velocidad de cracking | GPU compatible | Seguridad actual | Uso real hoy                               | Comentario directo                                        |
|---------------|-----------------------|----------------|------------------|--------------------------------------------|-----------------------------------------------------------|
| MD5           | Muy rápido            | Si             | Muy debil        | Casi abandonado                            | Se rompe en segundos. Nunca debe usarse.                  |
| SHA1          | Muy rápido            | Si             | Debil            | Muy poco usado                             | Ataques reales lo rompen. No sirve para contraseñas.      |
| SHA256        | Rápido                | Si             | Medio-bajo       | Usado para integridad, no para contraseñas | No es suficiente para contraseñas porque es muy rápido.   |
| SHA512        | Rápido                | Si             | Medio-bajo       | Linux antiguo (sha512crypt)                | Mejor que SHA256, pero aun rápido comparado con modernos. |
| bcrypt        | Lento                 | Limitado       | Fuerte           | Muy usado en web; WordPress, Django, etc   | Resiste GPU. Diseño moderno y seguro.                     |
| yescrypt      | Muy lento             | No             | Muy fuerte       | Linux moderno (Ubuntu 22+)                 | Diseñado para frustrar cracking. Lo vimos en la prueba.   |
| Argon2        | Muy lento             | No/limitado    | Muy fuerte       | Estándar moderno recomendado               | Es el futuro. Memory-hard, anti GPU.                      |

### Como se relaciona con nuestro laboratorio

John y Hashcat funcionan perfecto con hashes viejos. Por eso rompimos MD5 y SHA1 al instante. Yescrypt fue un dolor y falló. Exactamente como debería pasar en la vida real.

Un atacante que robe */etc/shadow* con yescrypt no gana mucho. Si la clave del usuario es buena, no la va a romper.

La prueba en VM comprobó la diferencia ENTRE ERA VIEJA Y ERA MODERNA. Y eso es lo que nos hace entender ciberseguridad en serio, no solo pentesting.

## 6. Romper hashes clásicos de Metasploitable2

Metasploitable2 trae usuarios con contraseñas ridículas.

En Metasploitable:

```
sudo cat /etc/shadow
```

ahora para traer ese archivo a Kali desde Metasploitable2, hay que hacer un par de cosas... ya vimos que **scp** es una forma segura y simple, el único tema es que metasploitable2 es una maquina “viejita” y usa un servidor **ssh** que es considerado inseguro por Kali por lo que lo más seguro es que nos rechace el envío de archivos por scp de forma convencional, por lo que nos daremos una pequeña vuelta. Veamos:

En Kali cuando intentamos traer por scp de la forma convencional

```
scp msfadmin@192.168.118.129:/etc/shadow .
```

(ojo con el punto final que es importante y la dirección ip es la de la maquina metasploitable2)

Kali que utiliza OpenSSH moderno responde negativamente con:

```
(ozone@kali)-[~]  
$ scp msfadmin@192.168.118.129:/etc/shadow .  
Unable to negotiate with 192.168.118.129 port 22: no matching host key type found. Their offer: ssh-rsa,ssh-dss  
scp: Connection closed
```

¿Qué quiere decir esto?

Metasploitable2 solo sabe usar **ssh-rsa** y **ssh-dss** (algoritmos antiguos). Kali moderno los bloquea por seguridad por lo tanto rechaza la conexión.

Entonces haremos lo siguiente:

Vamos a forzar a que nuestro Kali “moderno” acepte esos algoritmos antiguos con:

```
scp -oHostKeyAlgorithms=+ssh-rsa -oPubkeyAcceptedAlgorithms=+ssh-rsa  
msfadmin@192.168.118.129:/etc/shadow .
```

```
(ozone@kali)~$  
$ scp -oHostKeyAlgorithms=+ssh-rsa -oPubkeyAcceptedAlgorithms=+ssh-rsa msfadmin@192.168.118.129:/etc/shadow .  
The authenticity of host '192.168.118.129 (192.168.118.129)' can't be established.  
RSA key fingerprint is: SHA256:BQ  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '192.168.118.129' (RSA) to the list of known hosts.  
** WARNING: connection is not using a post-quantum key exchange algorithm.  
** This session may be vulnerable to "store now, decrypt later" attacks.  
** The server may need to be upgraded. See https://openssh.com/pq.html  
msfadmin@192.168.118.129's password:  
scp: remote open "/etc/shadow": Permission denied
```

Luego de conectar nos aparece finalmente el mensaje de “**Permiso denegado**”. ¿Por qué? Y esto es parte del aprendizaje.

Básicamente no tenemos permisos o privilegios suficientes para ver o copiar nada de esa carpeta. Así que haremos lo siguiente:

En la maquina **Metasploitable2** copiaremos el contenido de `/etc/shadow` a la carpeta temporal y le daremos permisos **644**

| Número | Binario | Lectura (r) | Escritura (w) | Ejecución (x) |
|--------|---------|-------------|---------------|---------------|
| 0      | 000     | ✗           | ✗             | ✗             |
| 1      | 001     | ✗           | ✗             | ✓             |
| 2      | 010     | ✗           | ✓             | ✗             |
| 3      | 011     | ✗           | ✓             | ✓             |
| 4      | 100     | ✓           | ✗             | ✗             |
| 5      | 101     | ✓           | ✗             | ✓             |
| 6      | 110     | ✓           | ✓             | ✗             |
| 7      | 111     | ✓           | ✓             | ✓             |

```
msfadmin@metasploitable:~$ sudo cp /etc/shadow /tmp/shadow  
msfadmin@metasploitable:~$ sudo chmod 644 /tmp/shadow  
msfadmin@metasploitable:~$ _
```

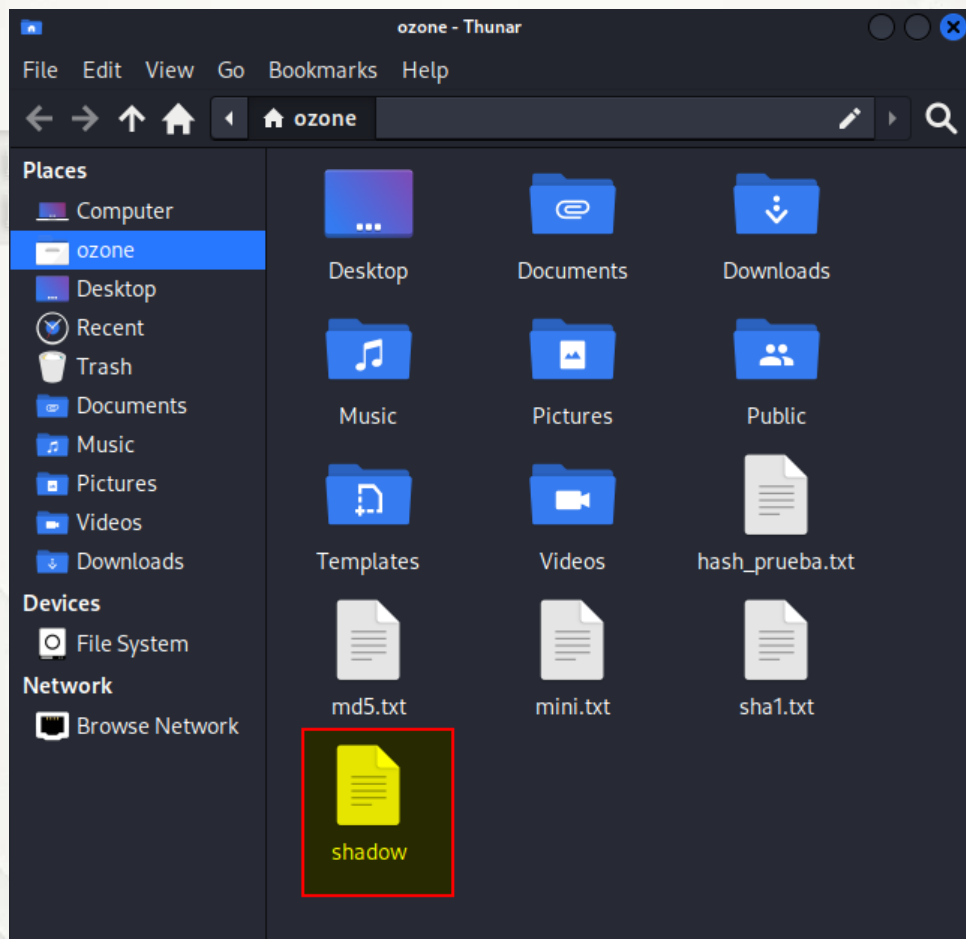
Ahora el archivo es accesible por cualquier usuario.

Y desde Kali ahora lo podemos copiar:

```
scp -oHostKeyAlgorithms=+ssh-rsa -oPubkeyAcceptedAlgorithms=+ssh-rsa  
msfadmin@192.168.118.129:/tmp/shadow .
```

```
(ozone@kali)~$  
$ scp -oHostKeyAlgorithms=+ssh-rsa -oPubkeyAcceptedAlgorithms=+ssh-rsa msfadmin@192.168.118.129:/tmp/shadow .  
** WARNING: connection is not using a post-quantum key exchange algorithm.  
** This session may be vulnerable to "store now, decrypt later" attacks.  
** The server may need to be upgraded. See https://openssh.com/pq.html  
msfadmin@192.168.118.129's password:  
shadow 100% 1207 376.9KB/s 00:00
```

Y ya tenemos nuestro archivo copiado desde una maquina con algoritmos antiguos a nuestro Kali “moderno y seguro”.





Ahora copiaremos 3 hashes desde el archivo **shadow**... el que por defecto trae varios usuarios.

```
(ozone@kali) [~]
$ cat shadow
root:$1$/avpfBJ1$x0z8w5UF9Iv./DR9E9Lid.:14747:0:99999:7:::
daemon*:14684:0:99999:7:::
bin*:14684:0:99999:7:::
sys:$1$fUX6BP0t$MiyC3Up0zQJqz4s5wFD9l0:14742:0:99999:7:::
sync*:14684:0:99999:7:::
games*:14684:0:99999:7:::
man*:14684:0:99999:7:::
lp*:14684:0:99999:7:::
mail*:14684:0:99999:7:::
news*:14684:0:99999:7:::
uucp*:14684:0:99999:7:::
proxy*:14684:0:99999:7:::
www-data*:14684:0:99999:7:::
backup*:14684:0:99999:7:::
list*:14684:0:99999:7:::
irc*:14684:0:99999:7:::
gnats*:14684:0:99999:7:::
nobody*:14684:0:99999:7:::
libuuid!:14684:0:99999:7:::
dhcp*:14684:0:99999:7:::
syslog*:14684:0:99999:7:::
klog:$1$f2ZVMS4K$R9XkI.CmLdHhdUE3X9jqP0:14742:0:99999:7:::
sshd*:14684:0:99999:7:::
msfadmin:$1$XN10Zj2c$Rt/zzCW3mLtUWA.ihZjA5/:14684:0:99999:7:::
bind*:14685:0:99999:7:::
postfix*:14685:0:99999:7:::
ftp*:14685:0:99999:7:::
postgres:$1$Rw35ik.x$MgQgZUu05pAoUvfJhfcYe/:14685:0:99999:7:::
mysql!:14685:0:99999:7:::
tomcat55*:14691:0:99999:7:::
distccd*:14698:0:99999:7:::
user:$1$HESu9xrH$k.o3G93DGoXIiQKkPmUgZ0:14699:0:99999:7:::
service:$1$kR3ue7JZ$7GxELDpr50hp6cjZ3Bu//:14715:0:99999:7:::
telnetd*:14715:0:99999:7:::
proftpd!:14727:0:99999:7:::
statd*:15474:0:99999:7:::
```

Tomaremos 2 o 3 hashes (usuarios como msfadmin, user y service) en un archivo.txt.

```
grep -E "msfadmin|user|service" shadow > meta_hashes.txt
```

```
(ozone@kali) [~]
$ grep -E "msfadmin|user|service" shadow > meta_hashes.txt
```

Podemos ver como nuestro nuevo archivo contiene los 3 hashes correspondientes a los 3 usuarios solicitados por nosotros:

```
(ozone@kali)-[~]  
$ cat meta_hashes.txt  
msfadmin:$1$XN10Zj2c$Rt/zzCW3mLtUWA.iHZjA5/:14684:0:99999:7 :::  
user:$1$HESu9xrH$k.o3G93DGoXIiQKkPmUgZ0:14699:0:99999:7 :::  
service:$1$kR3ue7JZ$7GxELDupr50hp6cjZ3Bu//:14715:0:99999:7 :::
```

Ahora... a romper todo:

```
john metas_hashes.txt
```

Veremos como John rompe casi todas en segundos.

```
(ozone@kali)-[~]  
$ john meta_hashes.txt  
Warning: detected hash type "md5crypt", but the string is also recognized as "md5crypt-long"  
Use the "--format=md5crypt-long" option to force loading these as that type instead  
Using default input encoding: UTF-8  
Loaded 3 password hashes with 3 different salts (md5crypt, crypt(3) $1$ (and variants) [MD5 256/256 AVX2 8x3])  
Will run 4 OpenMP threads  
Proceeding with single, rules:Single  
Press 'q' or Ctrl-C to abort, almost any other key for status  
service (service)  
msfadmin (msfadmin)  
user (user)  
3g 0:00:00:00 DONE 1/3 (2026-02-25 18:16) 300.0g/s 28500p/s 28800c/s 28800C/s user..msfadmin  
Use the "--show" option to display all of the cracked passwords reliably  
Session completed.
```

Esto confirma dos cosas importantes:

1. **Son hashes md5crypt**

Este formato es viejo y muy fácil de romper con hardware moderno.

2. **Las contraseñas de Metasploitable2 son debilísimas**

Metasploitable es intencionalmente vulnerable, por eso John las revienta en nada con el modo **single** (ni siquiera uso fuerza bruta completa).

## 7. Ataque con diccionario específico

Usaremos el diccionario más famoso:

`/usr/share/wordlists/rockyou.txt`

Ejemplo:

`john metas_hashes.txt --wordlist=/usr/share/wordlists/rockyou.txt`

```
(ozone@kali)-[~]
$ john metas_hashes.txt --wordlist=/usr/share/wordlists/rockyou.txt
Warning: detected hash type "md5crypt", but the string is also recognized as "md5crypt-long"
Use the "--format=md5crypt-long" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 3 password hashes with 3 different salts (md5crypt, crypt(3) $1$ (and variants) [MD5 256/256 AVX2 8x3])
No password hashes left to crack (see FAQ)
```

Podemos ver los resultados con:

`john --show metas_hashes.txt`

```
(ozone@kali)-[~]
$ john --show metas_hashes.txt
msfadmin:msfadmin:14684:0:99999:7:::
user:user:14699:0:99999:7:::
service:service:14715:0:99999:7:::

3 password hashes cracked, 0 left
```

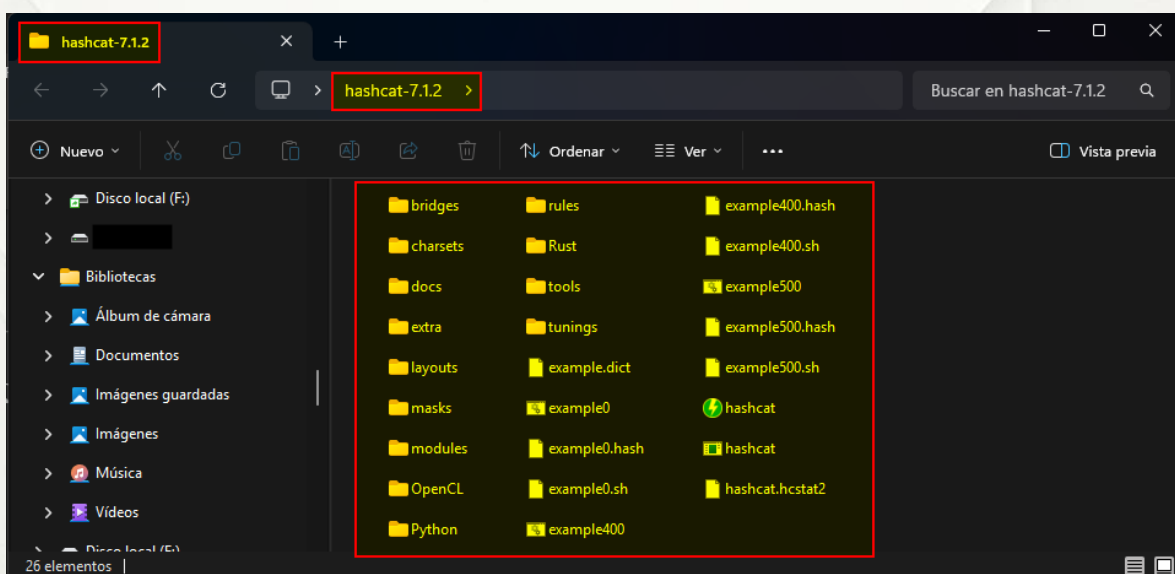
Esto acelera ataques por fuerza bruta realista.

## 6. Ataque avanzado con Hashcat desde Windows

Para instalar hashcat en Windows, vamos a la página oficial <https://hashcat.net/hashcat/> y descargamos el archivo que dice “hashcat binaries”.

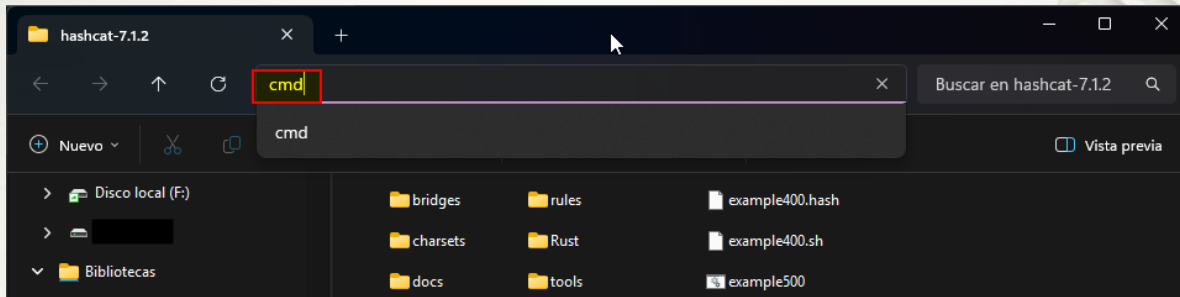


Hashcat en Windows no es un programa que se instale, es un “*portable*” por lo que simplemente lo descomprimiremos en el Escritorio

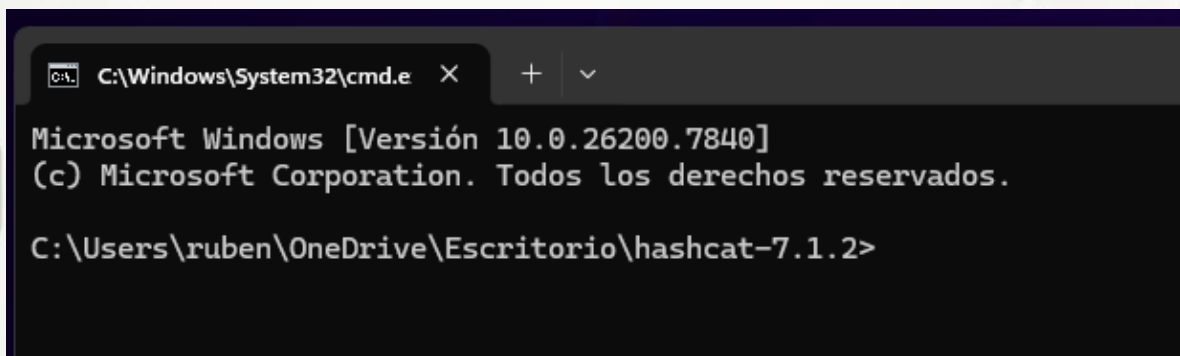




Haremos click derecho en la barra de dirección y escribiremos cmd



Con esto nos abrirá la consola de comandos de Windows con la ruta ya lista en nuestra carpeta:



Probamos que hashcat inicia con  
hashcat.exe -I

```
Microsoft Windows [Versión 10.0.26200.7840]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\ruben\OneDrive\Escritorio\hashcat-7.1.2>hashcat.exe -I
hashcat (v7.1.2) starting in backend information mode

Successfully initialized the NVIDIA main driver CUDA runtime library.

Failed to initialize NVIDIA RTC library.

* Device #1: CUDA SDK Toolkit not installed or incorrectly installed.
  CUDA SDK Toolkit required for proper device support and utilization.
  For more information, see: https://hashcat.net/faq/wrongdriver
  Falling back to OpenCL runtime.

If you are using WSL2 you can use CUDA instead of OpenCL.
Users must not install any NVIDIA GPU Linux driver within WSL 2
For all details: https://docs.nvidia.com/cuda/wsl-user-guide/index.html

TLDR; go to https://developer.nvidia.com/cuda-downloads and follow this path:
  Linux -> Architecture -> Distribution -> Version -> deb (local)
Follow the installation Instructions on the website.

OpenCL Info:
=====

OpenCL Platform ID #1
Vendor..: NVIDIA Corporation
Name....: NVIDIA CUDA
Version.: OpenCL 3.0 CUDA 12.6.65

Backend Device ID #01
Type.....: GPU
Vendor.ID.....: 32
Vendor.....: NVIDIA Corporation
Name.....: NVIDIA GeForce GTX 1080 Ti
Version.....: OpenCL 3.0 CUDA
Processor(s)...: 28
Preferred.Thrd.: 32
Clock.....: 1645
```

Hashcat inicia correctamente. Como podemos ver a diferencia de lo que ocurría en la máquina virtual Kali Linux en VMware y como se explicó previamente, en Kali, la herramienta hashcat no tenía acceso a la GPU, que es el motivo de esta prueba en Windows, por tratarse de una máquina virtual.

Ahora vamos a generar un hash de prueba. Para esto en Kali generamos con:

```
mkpasswd --method=sha512crypt Gato2024
```

```
(ozone@kali)-[~]  
$ mkpasswd --method=sha512crypt Gato2024  
$6$uJ0gMQz9pw62vwhA$E//IBzV0/gasvSvUbxUp0sJu0sgv.14egQIxI0dUeZol90Wxn3iw7qqDCm9YJX4edw.vIoskA/vcrbfzGBTk/
```

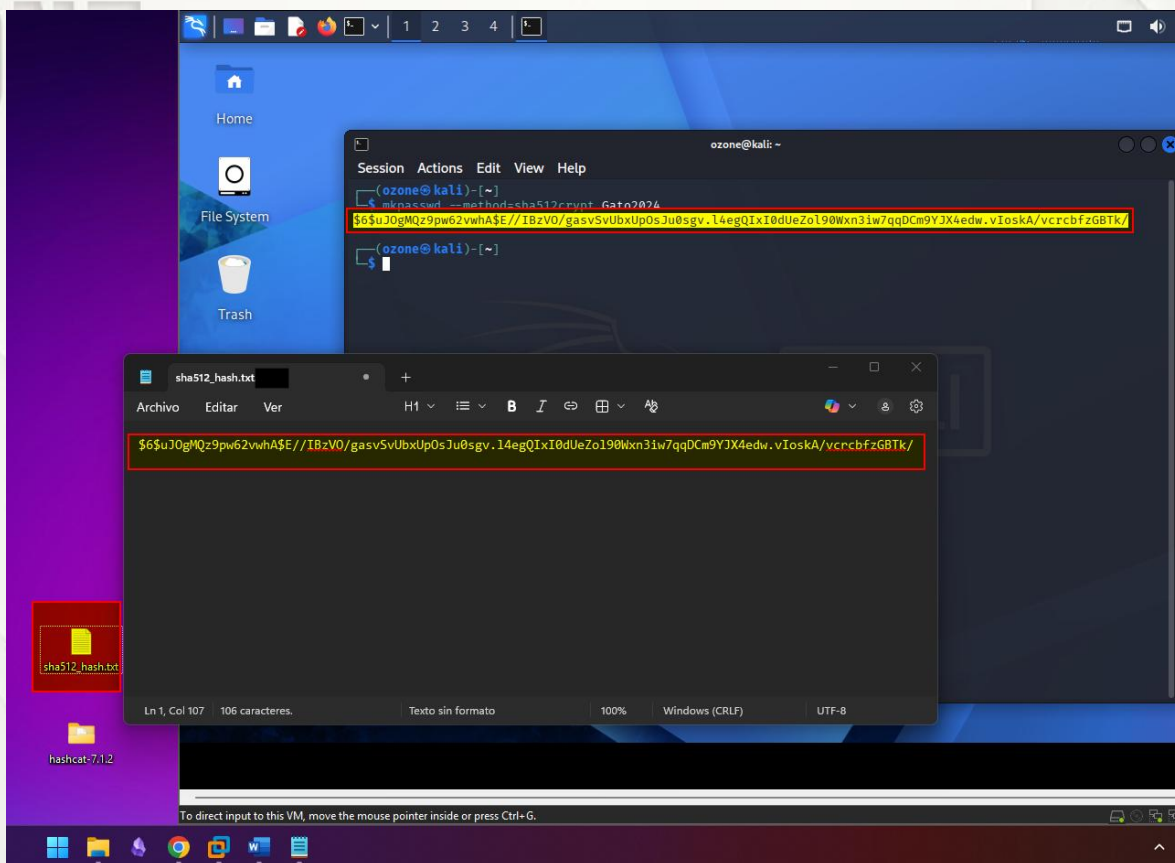
Y copiamos ese valor y lo pegamos en un archivo de texto en Windows.

**Clave real: Gato2024**

Creamos un archivo de texto en Windows:

**sha512\_hash.txt**

Y pegamos solo esa línea tal cual muestra la imagen a continuación.



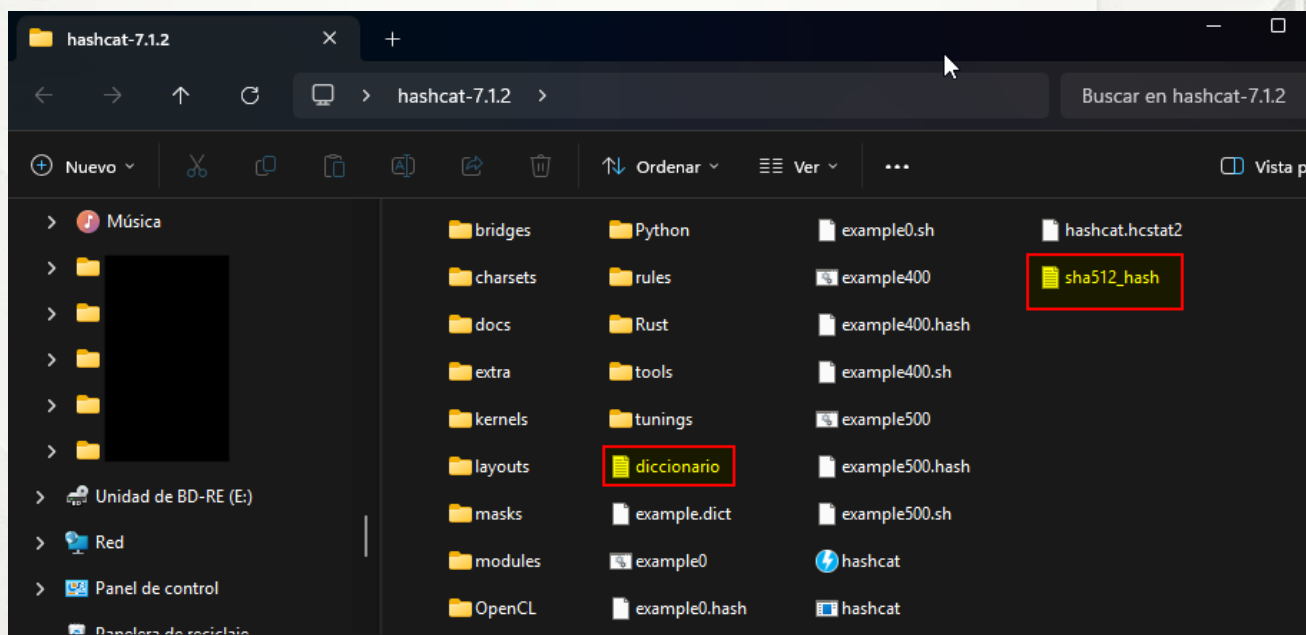
A continuación vamos a crear un diccionario personalizado; podemos copiar el que está en el siguiente link en un archivo de texto y añadir en algún punto nuestra contraseña personalizada <https://contest-2010.korelogic.com/wordlists/ArabicNames.dic> la idea es que este sea optimizado, pequeño y realista pensado para que:

- Hashcat lo pueda cargar rápido
- Sea creíble como diccionario real
- Que incluya *Gato2024* sin estar al principio
- Funcione perfecto con la GPU
- Demore menos de 5 minutos en romper *SHA512crypt* (modo 1800)

Podemos copiar y pegar esto en un archivo llamado:

*diccionario.txt*

y ahora dejamos ambos archivos dentro de la carpeta donde está hashcat (la que descomprimimos)





Y ahora desde la consola (*haciendo click derecho en la barra de búsqueda como se explicó previamente*) ejecutamos hashcat de la siguiente forma:

```
hashcat.exe -m 1800 sha512_hash.txt diccionario.txt -O
```

```
C:\Users\ruben\OneDrive\Escritorio\hashcat-7.1.2>hashcat.exe -m 1800 sha512_hash.txt diccionario.txt -O
hashcat (v7.1.2) starting

Successfully initialized the NVIDIA main driver CUDA runtime library.
Failed to initialize NVIDIA RTC library.

* Device #1: CUDA SDK Toolkit not installed or incorrectly installed.
  CUDA SDK Toolkit required for proper device support and utilization.
  For more information, see: https://hashcat.net/faq/wrongdriver
  Falling back to OpenCL runtime.

If you are using WSL2 you can use CUDA instead of OpenCL.
Users must not install any NVIDIA GPU Linux driver within WSL 2
For all details: https://docs.nvidia.com/cuda/wsl-user-guide/index.html

TLDR; go to https://developer.nvidia.com/cuda-downloads and follow this path:
  Linux -> Architecture -> Distribution -> Version -> deb (local)
Follow the installation Instructions on the website.

OpenCL API (OpenCL 3.0 CUDA 12.6.65) - Platform #1 [NVIDIA Corporation]
=====
* Device #01: NVIDIA GeForce GTX 1080 Ti, 11263/11263 MB (2815 MB allocatable), 28MCU
=====

Minimum password length supported by kernel: 0
```

Como primer detalle vemos como es la GPU la que se hace cargo.

```
Approaching final keyspace - workload adjusted.
$6$uJ0gMQz9pw62vwhA$E//IBzVO/gasvSvUbxUp0sJu0sgv.l4egQIXI0dUeZol90Wxn3iw7qqDCm9YJX4edw.vIoskA/vcrbfbzGBTk/ Gato2024

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 1800 (sha512crypt $6$, SHA512 (Unix))
Hash.Target.....: $6$uJ0gMQz9pw62vwhA$E//IBzVO/gasvSvUbxUp0sJu0sgv.l4...zGBTk/
Time.Started.....: Wed Feb 25 22:39:58 2026 (0 secs)
Time.Estimated....: Wed Feb 25 22:39:58 2026 (0 secs)
Kernel.Feature...: Optimized Kernel (password length 0-15 bytes)
Guess.Base.....: File (diccionario.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#01.....: 954 H/s (1.14ms) @ Accel:12 Loops:62 Thr:512 Vec:1
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 96/96 (100.00%)
Rejected.....: 0/96 (0.00%)
Restore.Point....: 0/96 (0.00%)
Restore.Sub.#01..: Salt:0 Amplifier:0-1 Iteration:4960-5000
Candidate.Engine.: Device Generator
Candidates.#01...: Password1 -> MiPassword2024
Hardware.Mon.#01.: Temp: 59c Fan: 0% Util: 94% Core:1911MHz Mem:5005MHz Bus:16
```

Una vez terminado podemos ver que ha descubierto nuestra clave **Gato2024** y el tiempo fue de menos de un segundo, pero el valor que nos importa es la velocidad **954 H/s**. Se debe señalar en este sentido que el diccionario que utilizamos era bastante pequeño en comparación a diccionarios como rockyou y por eso el poco tiempo sumado al uso de GPU sobre CPU.

Para poder comparar haremos lo mismo en Kali con el mismo archivo, mismo diccionario pero esta vez por CPU por las razones que ya se han explicado.

```
(ozone@kali)-[~]  
$ mkpasswd --method=sha512crypt Gato2024  
$6$uJ0gMQz9pw62vwhA$E//IBzV0/gasvSvUbxUp0sJu0sgv.l4egQIxI0dUeZol90Wxn3iw7qqDCm9YJX4edw.vIoskA/vcrbfzGBTk/  
  
(ozone@kali)-[~]  
$ nano sha512_hash.txt  
  
(ozone@kali)-[~]  
$ nano diccionario.txt
```

Con todo listo ejecutamos

```
hashcat -m 1800 sha512_hash.txt diccionario.txt --force
```

```
$ hashcat -m 1800 sha512_hash.txt diccionario.txt --force  
hashcat (v7.1.2) starting  
  
You have enabled --force to bypass dangerous warnings and errors!  
This can hide serious problems and should only be done when debugging.  
Do not report hashcat issues encountered when using --force.  
  
OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_D  
EBUG) - Platform #1 [The pocl project]  
  
* Device #01: cpu-haswell-Intel(R) Core(TM) i9-10900KF CPU @ 3.70GHz, 2930/5861 MB (1024 MB allocatable), 4MCU
```

A diferencia del anterior vemos como es la CPU la que se hace cargo.

```
Approaching final keypace - workload adjusted.
$6$uJ0gMQz9pw62vwhA$E//IBzV0/gasvSvUbxUp0sJu0sgv.l4egQIXI0dUeZol90Wxn3iw7qqDCm9YJX4edw.vIoskA/vcrcbfzGBTk/:Gat
o2024
Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 1800 (sha512crypt $6$, SHA512 (Unix))
Hash.Target.....: $6$uJ0gMQz9pw62vwhA$E//IBzV0/gasvSvUbxUp0sJu0sgv.l4 ... zGBTk/
Time.Started.....: Wed Feb 25 22:53:37 2026, (1 sec)
Time.Estimated...: Wed Feb 25 22:53:38 2026, (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.....: File (diccionario.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#01.....: 142 H/s (6.89ms) @ Accel:41 Loops:1000 Thr:1 Vec:4
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 96/96 (100.00%)
Rejected.....: 0/96 (0.00%)
Restore.Point...: 0/96 (0.00%)
Restore.Sub.#01..: Salt:0 Amplifier:0-1 Iteration:4000-5000
Candidate.Engine.: Device Generator
Candidates.#01...: Password1 → MiPassword2024
Hardware.Mon.#01.: Util: 25%

Started: Wed Feb 25 22:53:10 2026
Stopped: Wed Feb 25 22:53:39 2026
```

Podemos ver que también se ha demorado muy poco, esto insisto por el diccionario, pero el valor que nos interesa para esta parte es la velocidad: **142 H/s**.

**¿Qué quiere decir todo esto? ¿No es una exageración considerando que entre ambas hay una diferencia de un segundo?**

Y la respuesta es que no, no es una exageración decir que para romper claves lo mejor es utilizar GPU y esta es la **evidencia**, más allá del tiempo de esta prueba.

Este resultado confirma exactamente lo que quería demostrar: la diferencia **brutal** entre **CPU** y **GPU** al romper hashes fuertes (sha512crypt).

Veamos el análisis:

## 1. Velocidad en Kali con CPU

*Speed.#01: 142 H/s*

Eso significa:

**142 intentos por segundo rompiendo sha512crypt**

Esto es normal, sha512crypt es *lento por diseño*.

Tiempo total real: 1 segundo (porque el diccionario utilizado es pequeño)

## 2. Velocidad en Windows con GPU 1080Ti

Speed.#01: **954 H/s**

Eso es:

7 veces más rápido que la CPU en Kali ( $954 \div 142 = 6.71$ )

## 3. Comparación directa

| Plataforma    | Velocidad | Diferencia    |
|---------------|-----------|---------------|
| CPU Kali      | 142 H/s   | Base          |
| Nvidia 1080Ti | 954 H/s   | 7x más rápido |

La GPU destroza a la CPU, incluso con un hash lento como sha512crypt.

## 4. ¿Que demuestra esto?

- CPU cracking no sirve para diccionarios grandes Rockyou: 14.3 millones de entradas CPU a **142 H/s** demoraría aproximadamente **28 horas** solo para rockyou.
- GPU es indispensable para cracking moderno 1080Ti a **954 H/s** demoraría **4 horas y media** para rockyou.
- sha512crypt sigue siendo muy resistente. Aunque GPU acelera, sigue siendo lento comparado con hashes antiguos.
- Toda herramienta funciona bien si el formato del hash es compatible. Los fallos de la sección 3 fueron por yescrypt, no por John o Hashcat.

El equipo considera una tarjeta de video que si bien es buena ni siquiera es de última generación como las ultimas RTX. Así que de paso entendemos porque el precio de este ítem de nuestros computadores está tan disparado, es su capacidad de procesamiento y las empresas de iA están demandando mucha de esta potencia.



**Tabla comparativa Hashcat – CPU vs 1080 Ti vs 4090**

| Modo / Algoritmo                  | i9-10900KF<br>(CPU) | GTX 1080<br>Ti | RTX 4090   | Comentario directo                                                        |
|-----------------------------------|---------------------|----------------|------------|---------------------------------------------------------------------------|
| <b>MD5 (modo 0)</b>               | 55 MH/s             | 30 GH/s        | 150 GH/s   | GPU destruye CPU por factor 500–3000.                                     |
| <b>SHA1 (modo 100)</b>            | 18 MH/s             | 9 GH/s         | 60 GH/s    | SHA1 sigue siendo rápido en GPU.                                          |
| <b>SHA256 (modo 1400)</b>         | 6 MH/s              | 3 GH/s         | 35 GH/s    | CPU totalmente irrelevante.                                               |
| <b>SHA512 (modo 1700)</b>         | 2 MH/s              | 1.5 GH/s       | 15 GH/s    | Algoritmo más pesado pero GPU igual gana fácil.                           |
| <b>NTLM (modo 1000)</b>           | 90 MH/s             | 70 GH/s        | 280 GH/s   | 4090 lo revienta casi todo en segundos.                                   |
| <b>NetNTLMv2 (modo 5600)</b>      | 350 kH/s            | 350 MH/s       | 2.5 GH/s   | Diferencia brutal, especialmente contra CPU.                              |
| <b>bcrypt cost 05 (modo 3200)</b> | 120 H/s             | 1.000 H/s      | 6.500 H/s  | bcrypt es lento para todos; aun así la 4090 es 50x más rápido que la CPU. |
| <b>sha512crypt (modo 1800)</b>    | 140 H/s             | 900 H/s        | 12.000 H/s | El que usé: CPU es inútil para bruteforce real.                           |

## Resumen directo

Velocidad y diferencia aproximada:

| Comparación | CPU vs 1080 Ti | CPU vs 4090 |
|-------------|----------------|-------------|
| MD5         | 1x → 550x      | 1x → 2700x  |
| SHA1        | 1x → 500x      | 1x → 3500x  |
| SHA256      | 1x → 500x      | 1x → 6000x  |
| NTLM        | 1x → 800x      | 1x → 3000x  |
| NetNTLMv2   | 1x → 1000x     | 1x → 7000x  |
| bcrypt      | 1x → 8x        | 1x → 50x    |
| sha512crypt | 1x → 6x        | 1x → 85x    |

## Resumen directo con horas aproximadas

| Comparacion        | CPU i9-10900KF | GTX 1080 Ti | RTX 4090   | Horas CPU              | Horas 1080 Ti           | Horas 4090          |
|--------------------|----------------|-------------|------------|------------------------|-------------------------|---------------------|
| MD5                | 55 MH/s        | 30 GH/s     | 150 GH/s   | 1.6 h                  | ~0.01 h (36 seg)        | ~0.002 h (7 seg)    |
| SHA1               | 18 MH/s        | 9 GH/s      | 60 GH/s    | 4.8 h                  | ~0.03 h (108 seg)       | ~0.0015 h (5 seg)   |
| SHA256             | 6 MH/s         | 3 GH/s      | 35 GH/s    | 14.3 h                 | ~0.10 h (6 min)         | ~0.0025 h (9 seg)   |
| SHA512             | 2 MH/s         | 1.5 GH/s    | 15 GH/s    | 43 h                   | ~0.20 h (12 min)        | ~0.006 h (22 seg)   |
| NTLM               | 90 MH/s        | 70 GH/s     | 280 GH/s   | 1 h                    | ~0.004 h (14 seg)       | ~0.001 h (4 seg)    |
| NetNTLMv2          | 350 kH/s       | 350 MH/s    | 2.5 GH/s   | 245 h<br>(10 días)     | ~0.24 h<br>(14 min)     | ~0.034 h<br>(2 min) |
| bcrypt cost 05     | 120 H/s        | 1000 H/s    | 6500 H/s   | 30.000 h<br>(3.4 años) | ≈3.000 h<br>(4 meses)   | ≈450 h<br>(19 días) |
| sha512crypt (1800) | 140 H/s        | 900 H/s     | 12.000 H/s | 24.000 h<br>(2.7 años) | ~3.200 h<br>(4.5 meses) | ~240 h<br>(10 días) |

### Los algoritmos rápidos (MD5, SHA1, SHA256, NTLM):

La diferencia es **absurda**. Una prueba de 6 caracteres:

- CPU puede tardar horas.
- 1080 Ti: segundos o minutos.
- 4090: segundos.

### Algoritmos lentos (bcrypt, sha512crypt):

- Están diseñados para ser lentos.
- Incluso una 4090 tarda días o semanas en bruteforce completo.
- En CPU: **años**.



## Conclusión

La culminación de este proyecto de laboratorio permite extraer valiosas lecciones de carácter técnico y estratégico, tanto para el ejercicio del *Pentesting* (Red Team) como para la fortificación de infraestructuras (Blue Team). En primera instancia, quedó demostrada la efectividad de herramientas de auditoría como *John the Ripper* y *Hashcat*, pero condicionado estrictamente al conocimiento profundo de los formatos y variantes de cada *hash*, evidenciando que las limitaciones iniciales no se debían a fallos del software, sino a las restricciones propias de la arquitectura del algoritmo evaluado.

El experimento con la variante moderna de *yescrypt* implementada en las versiones recientes de Ubuntu ratifica el éxito de la ingeniería de seguridad actual. Al tratarse de un algoritmo diseñado específicamente para ser *memory-hard* y *CPU-hard*, neutraliza la ventaja competitiva de las aceleradoras gráficas y ralentiza drásticamente los ataques de fuerza bruta o diccionario. Esto nos enseña que, si un atacante logra comprometer el archivo `/etc/shadow`, el factor crítico de seguridad se traslada inmediatamente a la robustez de la contraseña del usuario; una clave débil como "cat88" cae de forma instantánea ante un diccionario personalizado, mientras que una credencial alfanumérica compleja bajo este algoritmo requeriría un esfuerzo computacional impracticable en la actualidad.

Por último, la comparativa de rendimiento entre plataformas evidenció el impacto de la virtualización sobre el hardware de alto rendimiento. Las limitaciones de los hipervisores convencionales para realizar un *passthrough* directo de la GPU confinan a las máquinas virtuales al uso exclusivo de la CPU, reduciendo la tasa de hashes por segundo de forma dramática. Los datos obtenidos entre la CPU y las GPUs Nvidia (GTX 1080 Ti y RTX 4090) confirman que, para escenarios reales de cracking a gran escala sobre algoritmos tradicionales (como los analizados en la máquina Metasploitable2), el uso de entornos nativos optimizados para GPU no es una opción secundaria, sino un requisito técnico indispensable. En conclusión, el laboratorio no solo valida la mecánica del cracking, sino que subraya la importancia imperativa de migrar hacia funciones de derivación de claves modernas (*yescrypt*, *Argon2*, *bcrypt*) para salvaguardar la identidad digital en entornos corporativos y de producción.